



Implementing a unified solver for nonlinearly constrained optimization

Charlie Vanaret^{1,2} · Sven Leyffer¹

Received: 19 June 2024 / Accepted: 22 February 2026
© The Author(s) 2026

Abstract

SQP and interior-point methods (also referred to as Lagrange-Newton methods) typically share key algorithmic components, such as strategies for computing descent directions and mechanisms that promote global convergence. Building on this insight, we introduce a unifying framework with eight building blocks that abstracts the workflows of Lagrange-Newton methods. We then present Uno, a modular C++ solver that implements our unifying framework and allows the automatic combination of a wide range of strategies with no programming effort from the user. Uno is meant to (1) organize mathematical optimization strategies into a coherent hierarchy; (2) offer a wide range of efficient and robust methods that can be compared for a given instance; (3) enable researchers to experiment with novel optimization strategies; and (4) reduce the cost of development and maintenance of multiple optimization solvers. Uno's software design allows user to compose new customized solvers for emerging optimization areas such as robust optimization or optimization problems with complementarity constraints, while building on reliable nonlinear optimization techniques. We demonstrate that Uno is highly competitive against state-of-the-art solvers filter-SQP, IPOPT, SNOPT, MINOS, LANCELOT, LOQO, and CONOPT on a subset of 429 small problems from the CUTE collection. Uno is available as open-source software under the MIT license at <https://github.com/cvanaret/Uno> and via its C, Julia, Python, Fortran, and AMPL interfaces.

Keywords Nonlinearly constrained optimization · Interior-point methods · Sequential quadratic programming methods · Globalization techniques

Mathematics Subject Classification 49M15 · 65K05 · 90C30 · 90C51 · 90C55

✉ Charlie Vanaret
vanaret@zib.de

Sven Leyffer
leyffer@anl.gov

¹ Mathematics and Computer Science Division, Argonne National Laboratory, Lemont, IL 60439, USA

² Department of Mathematical Optimization, Zuse-Institut Berlin, Berlin 14195, Germany

1 Motivation and contributions

We consider nonlinearly constrained optimization problems of the form

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & l \leq \begin{Bmatrix} c(x) \\ Ax \\ x \end{Bmatrix} \leq u, \end{aligned} \quad (1)$$

where $x \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, $c : \mathbb{R}^n \rightarrow \mathbb{R}^{m_c}$, $A \in \mathbb{R}^{m_A \times n}$, and $l \in (\mathbb{R} \cup \{-\infty\})^{m_c+m_A+n}$ and $u \in (\mathbb{R} \cup \{\infty\})^{m_c+m_A+n}$. f and c may be nonconvex, which results in a nonconvex optimization problem. This formulation allows for unbounded variables and equality constraints and explicitly separates general nonlinear, linear, and bound constraints, enabling solvers to readily exploit this structure. However, for the sake of simplicity of this presentation and without loss of generality, we consider the problem in the following form:

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & c(x) = 0 \\ & x \geq 0, \end{aligned} \quad (\text{NLP})$$

where $x \in \mathbb{R}^n$, $f : \mathbb{R}^n \rightarrow \mathbb{R}$, and $c : \mathbb{R}^n \rightarrow \mathbb{R}^m$.

Most derivative-based iterative methods for nonlinearly constrained nonconvex optimization (e.g., [18, 28, 34, 39]) share common algorithmic components. Building on this insight, we introduce an abstract framework structured around generic building blocks that describes these methods in a unified fashion. We then present Uno (Unifying Nonlinear Optimization)¹, a modular open-source solver for nonlinearly constrained optimization that unifies numerous state-of-the-art methods and organizes existing strategies into a coherent hierarchy. Optimization strategies are implemented as unique software components (e.g., all line-search methods may be generated based on a single `LineSearch` class); these efficient and robust implementations can be combined at will and compared on a given instance. This allows users to experiment with new algorithmic ideas by building upon Uno's abstractions and interfaces to modeling languages and subproblem solvers.

Uno allows the automatic generation of various strategy combinations on the fly with no programming effort from the user. While all combinations do not lead to convergent methods, some of them result in efficient solvers that may not exist as software implementations. We demonstrate that Uno is competitive against state-of-the-art solvers on a subset of 429 CUTE [5] test problems translated into AMPL, while being extensible and lightweight. We believe that Uno has the potential to serve as an experimental laboratory for practitioners and optimizers and to accelerate research in nonlinearly constrained optimization. Our ultimate goal is to promote the extension of state-of-the-art nonlinear optimization

¹ Uno was first introduced at the ISMP 2018 conference under the name Argonot [45].

techniques to new classes of problems such as problems with equilibrium constraints (see, e.g., [21, 32, 33, 36, 40]) and nonlinear robust optimization (see, e.g., [35]).

This paper is organized as follows. In Section 2 we introduce our notation and discuss relevant optimality conditions. In Section 3 we introduce our unifying framework with generic building blocks for unifying derivative-based iterative methods. In Section 4 we briefly describe state-of-the-art optimization strategies through the prism of our unifying framework. In Section 5 we present the basic algorithmic design of Uno and show how various nonlinear optimization methods fit within the architecture. In Section 6 we illustrate with two concrete strategy combinations how the building blocks interact with one another. In Section 7 we provide preliminary numerical results and compare Uno against state-of-the-art solvers.

2 Notation and stationarity conditions

In this section we define our notation and state first-order optimality conditions of (NLP).

2.1 Notation

We start by defining the scaled Lagrangian or Fritz John function [6] of (NLP) at (x, y, z, π) :

$$\mathcal{L}_\pi(x, y, z) \stackrel{\text{def}}{=} \pi f(x) - y^T c(x) - z^T x = \pi f(x) - \sum_{j=1}^m y_j c_j(x) - \sum_{i=1}^n z_i x_i,$$

where y and $z \geq 0$ are the Lagrange multipliers of the general constraints $c(x) = 0$ and the bound constraints $x \geq 0$, respectively, and $\pi \geq 0$ is an objective multiplier that is introduced to handle infeasibility or lack of constraint qualification (CQ) [37] in a consistent way.

$\nabla_x \mathcal{L}_\pi(x, y, z)$ is the gradient of the scaled Lagrangian with respect to x :

$$\nabla_x \mathcal{L}_\pi(x, y, z) \stackrel{\text{def}}{=} \pi \nabla f(x) - \sum_{j=1}^m y_j \nabla c_j(x) - z.$$

$\nabla_{xx}^2 \mathcal{L}_\pi(x, y)$ is the Hessian of the scaled Lagrangian with respect to x :

$$\nabla_{xx}^2 \mathcal{L}_\pi(x, y) = \pi \nabla^2 f(x) - \sum_{j=1}^m y_j \nabla^2 c_j(x).$$

2.2 First-order stationarity conditions

We are primarily concerned with first-order stationary points. The first-order optimality conditions (aka Fritz John conditions) of problem (NLP) at a stationary point x^* state that there exist (π^*, y^*, z^*) such that

$$(\text{stationarity}) \quad \nabla_x \mathcal{L}_{\pi^*}(x^*, y^*, z^*) = 0 \quad (2a)$$

$$(\text{primal feasibility}) \quad c(x^*) = 0, \quad x^* \geq 0 \quad (2b)$$

$$(\text{dual feasibility}) \quad \pi^* \geq 0, \quad z_i^* \geq 0, \quad (\pi^*, y^*, z^*) \neq (0, 0, 0) \quad (2c)$$

$$(\text{complementarity}) \quad x_i^* z_i^* = 0, \quad \forall i \in \{1, \dots, n\}. \quad (2d)$$

If $\pi^* > 0$, the optimality conditions are equivalent to the KKT conditions, which can be recovered by scaling Equation (2a) by $1/\pi^*$. If $\pi^* = 0$, they characterize Fritz John points, that is, feasible points at which a constraint qualification is violated.

3 A unifying framework for nonlinearly constrained optimization

A local quadratic approximation of (NLP) at iteration k about the current primal-dual point is given by

$$\begin{aligned} \min_x \quad & \frac{1}{2}(x - x^{(k)})^T H^{(k)}(x - x^{(k)}) + (\nabla f^{(k)})^T (x - x^{(k)}) \\ \text{s.t.} \quad & c^{(k)} + (\nabla c^{(k)})^T (x - x^{(k)}) = 0 \\ & x \geq 0, \end{aligned} \quad (3)$$

where $H(x, y)$ is any approximation of $\nabla_{xx}^2 \mathcal{L}_{\pi}(x, y)$, $f^{(k)} \stackrel{\text{def}}{=} f(x^{(k)})$, $c^{(k)} \stackrel{\text{def}}{=} c(x^{(k)})$, $\nabla f^{(k)} \stackrel{\text{def}}{=} \nabla f(x^{(k)})$, $\nabla c^{(k)} \stackrel{\text{def}}{=} \nabla c(x^{(k)})$, and $H^{(k)} \stackrel{\text{def}}{=} H(x^{(k)}, y^{(k)})$. For convex equality-constrained problems, an iteration can be interpreted as taking a Newton step on the first-order optimality conditions of the problem, hence the name *Lagrange-Newton methods*.

In this section we introduce a unified view for describing Lagrange-Newton methods and argue that they can be assembled by combining eight generic building blocks or *ingredients*. These ingredients are organized in four layers:

- the **reformulation** layer:
 - a **constraint relaxation strategy** constructs feasible subproblems by relaxing the general constraints;
 - an **inequality handling method** handles the inequality constraints;
- the **subproblem** layer:
 - a **Hessian model** determines the approximation of the Lagrangian Hessian;
 - an **inertia correction strategy** corrects the inertia of the Lagrangian Hessian or of the KKT matrix;

- the **subproblem solver** layer:
 - a **subproblem solver** approximately solves the subproblem by exploiting its characteristics, such as convexity, presence of constraints, presence of bounds, and presence of inequality constraints;
- the **globalization** layer:
 - a **globalization strategy** determines whether a trial iterate makes sufficient progress toward a solution and accepts or rejects it; and,
 - a **globalization mechanism** controls the length of the direction and defines the recourse action taken when a trial iterate is rejected.

This coloring will be used throughout to illustrate how the layers and ingredients interact with one another within an optimization algorithm. The role of each ingredient is shown in the following abstract algorithm (Algorithm 1).

Algorithm 1: Abstract algorithm.

```

Data: initial point  $(x^{(0)}, y^{(0)}, z^{(0)})$ 
Set  $k \leftarrow 0$ 
while termination criteria at  $(x^{(k)}, y^{(k)}, z^{(k)})$  not met do
    repeat globalization mechanism
        Solve a (sequence of) feasible subproblem (s) that approximate(s) (NLP) at
         $(x^{(k)}, y^{(k)}, z^{(k)})$ 
        Assemble trial iterate  $(\hat{x}^{(k+1)}, \hat{y}^{(k+1)}, \hat{z}^{(k+1)})$ 
        until  $(\hat{x}^{(k+1)}, \hat{y}^{(k+1)}, \hat{z}^{(k+1)})$  is acceptable ;
    Update  $(x^{(k+1)}, y^{(k+1)}, z^{(k+1)}) \leftarrow (\hat{x}^{(k)}, \hat{y}^{(k)}, \hat{z}^{(k)})$ 
     $k \leftarrow k + 1$ 
Result:  $(x^{(k)}, y^{(k)}, z^{(k)})$ 
    
```

The inner loop (**repeat**) generates and solves a feasible subproblem (possibly a sequence of feasible subproblems) until a trial iterate is accepted by the globalization strategy, and the outer loop (**while**) generates a sequence of acceptable iterates until termination.

The “wheel of strategies” (Figure 1) organizes a wide range of strategies into a coherent hierarchy. The outer layer represents the optimization layers, the middle layer represents the ingredients, and the inner layer represents the strategies. Strategies that perform similar tasks within an optimization method are listed under the same ingredient (e.g., a line search and a trust-region method are both globalization mechanisms).

4 Unified view of state-of-the-art techniques

In this section we describe a set of strategies that fall into each of the following ingredients of our unifying framework: globalization strategies, constraint relaxation

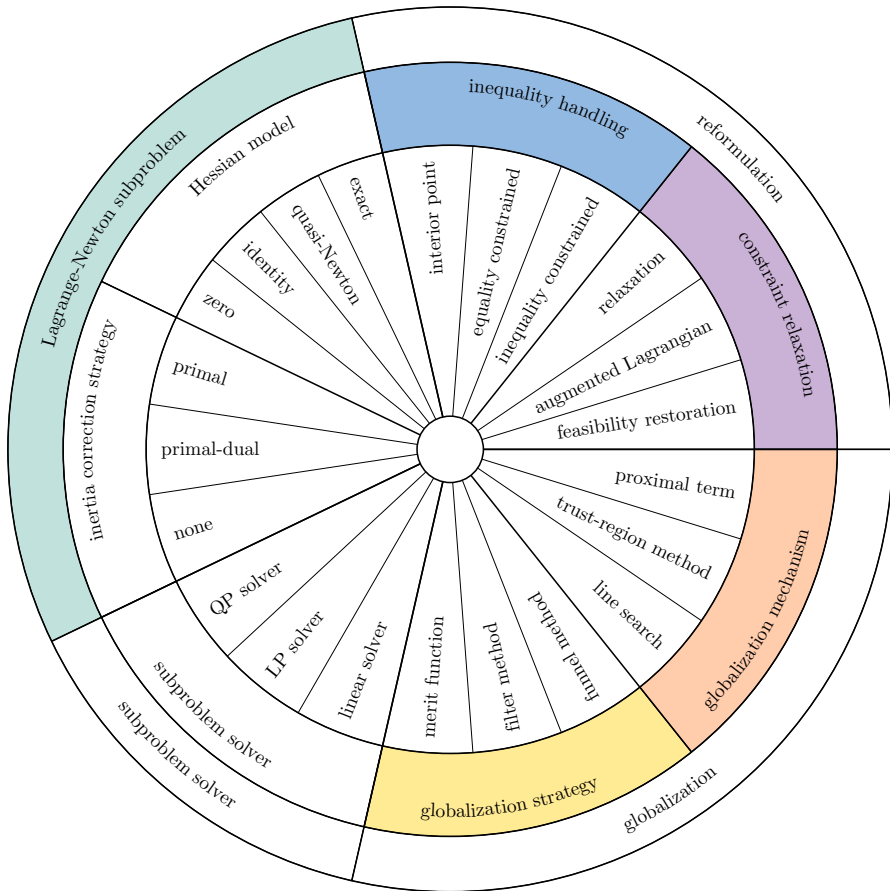


Fig. 1 Unifying framework: the “wheel of strategies”

strategies, inequality handling methods, Hessian models, inertia correction strategies, and globalization mechanisms. Our goal is to illustrate the wide variety of methods within a common notation to motivate the design of our modular solver.

4.1 Globalization strategies

Constrained optimization methods must achieve two competing goals: minimizing the objective function and minimizing the constraint violation. Globalization strategies determine whether a trial iterate $\hat{x}^{(k+1)} \stackrel{\text{def}}{=} x^{(k)} + \alpha d_x^{(k)}$ (for a given step length $\alpha \in (0, 1)$) makes acceptable progress with respect to these goals. We consider strategies that ensure *global convergence*, that is, convergence to a local minimum, or (weaker) stationary point, from any starting point. In addition, ideally, the minimization of the measure of infeasibility takes precedence. However, in problems where no feasible

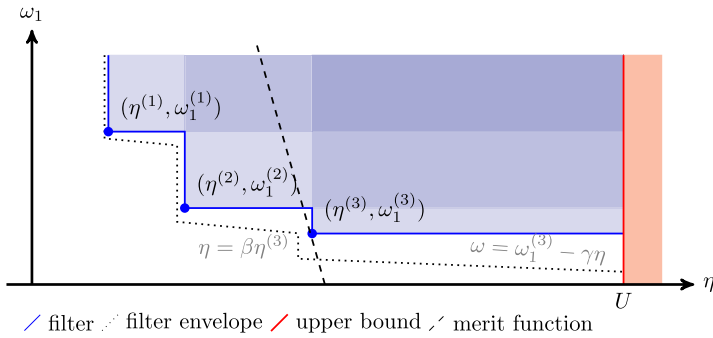


Fig. 2 Example of a filter and a merit function

point exists, a (global) minimum of the constraint violation is a certificate that the problem is infeasible.

Three (possibly primal-dual) *progress measures* are monitored throughout the optimization process:

1. an **infeasibility measure** η (typically $\|c(x)\|$ for some norm);
2. an **objective measure** ω_π parameterized by the objective multiplier $\pi \geq 0$ (typically $\pi f(x)$); and
3. an **auxiliary measure** ξ (terms such as barrier and proximal terms).

The local models of $\eta(x)$, $\omega_\pi(x)$, and $\xi(x)$ at iteration k about the current iterate are denoted by $\eta^{(k)}(d_x)$, $\omega_\pi^{(k)}(d_x)$, and $\xi^{(k)}(d_x)$ for a given primal direction d_x . We define the respective *predicted reductions*

$$\begin{aligned}\Delta\eta^{(k)}(d_x) &\stackrel{\text{def}}{=} \eta^{(k)}(0) - \eta^{(k)}(d_x) \\ \Delta\omega_\pi^{(k)}(d_x) &\stackrel{\text{def}}{=} \omega_\pi^{(k)}(0) - \omega_\pi^{(k)}(d_x) \\ \Delta\xi^{(k)}(d_x) &\stackrel{\text{def}}{=} \xi^{(k)}(0) - \xi^{(k)}(d_x)\end{aligned}$$

In order to ensure convergence, the progress measures and their local models must be intimately linked to the definitions of the problem and the subproblem (see Section 5.3.3).

The two main classes of globalization strategies are merit functions and filter methods (Figure 2). They typically enforce a sufficient decrease condition that forces some scalar combination of η , ω_π , and ξ to decrease by at least a fraction of the decrease predicted by the local model.

4.1.1 Merit functions

A merit (or penalty) function combines the three goals η , ω_ρ , and ξ into a single scalar value:

$$\psi_\rho(x) \stackrel{\text{def}}{=} \omega_\rho(x) + \eta(x) + \xi(x),$$

where $\rho \geq 0$ is an inverse penalty parameter. Its predicted reduction is given by

$$\Delta\psi_\rho^{(k)}(d_x) \stackrel{\text{def}}{=} \Delta\omega_\rho^{(k)}(d_x) + \Delta\xi^{(k)}(d_x) + \Delta\eta^{(k)}(d_x).$$

The trial iterate $\hat{x}^{(k+1)}$ is accepted if the actual reduction $\psi_\rho(x^{(k)}) - \psi_\rho(\hat{x}^{(k+1)})$ in the merit function is larger than a fraction $\sigma \in (0, 1)$ of its predicted reduction:

$$\psi_\rho(x^{(k)}) - \psi_\rho(\hat{x}^{(k+1)}) \geq \sigma \Delta\psi_\rho^{(k)}(\alpha d_x^{(k)}). \quad (4)$$

The sufficient decrease condition (4) (also known as Armijo condition) indicates how the subproblem solve is connected to the merit function to ensure global convergence.

4.1.2 Filter methods

Filter methods are motivated by the desire to decouple reduction in the objective function from progress toward feasibility. We can interpret a filter method as a mechanism to force iterates closer to the feasible region, so that unconstrained sufficient reduction conditions on the objective can be used to force convergence (see, e.g., [19, 22, 47, 48]). The decrease function is given by

$$\phi(x) \stackrel{\text{def}}{=} \omega_1(x) + \xi(x),$$

and its predicted reduction is given by

$$\Delta\phi^{(k)}(d_x) \stackrel{\text{def}}{=} \Delta\omega_1^{(k)}(d_x) + \Delta\xi^{(k)}(d_x).$$

Filter methods measure progress toward a solution by comparing the trial infeasibility measure η and objective measure ϕ to a filter $\mathcal{F}$, a list of pairs $(\eta^{(l)}, \phi^{(l)})$ (typically from previous iterates) such that no pair dominates another pair; that is, there exists no index l' such that $\eta^{(l')} < \eta^{(l)}$ and $\phi^{(l')} < \phi^{(l)}$ for all $(\eta^{(l)}, \phi^{(l)}) \in \mathcal{F}$. Formally, the trial iterate $\hat{x}^{(k+1)}$ is acceptable to the filter if and only if the following conditions hold:

$$\phi(\hat{x}^{(k+1)}) \leq \phi^{(l)} - \gamma\eta(\hat{x}^{(k+1)}) \quad \text{or} \quad \eta(\hat{x}^{(k+1)}) < \beta\eta^{(l)}, \quad \forall (\eta^{(l)}, \phi^{(l)}) \in \mathcal{F}^{(k)},$$

where $\gamma > 0$ and $0 < \beta < 1$ are constants that ensure that iterates cannot accumulate at infeasible limit points. These conditions are represented as the filter envelope in Figure 2.

The filter provides convergence only to a feasible limit because any infinite sequence of iterates must converge to a point where $\eta(x) = 0$, provided that $\phi(x)$ is bounded below. To ensure convergence to a local minimum, filter methods use a standard sufficient reduction (Armijo) condition from unconstrained optimization:

$$\phi(x^{(k)}) - \phi(\hat{x}^{(k+1)}) \geq \sigma \Delta\phi^{(k)}(\alpha d_x^{(k)}), \quad (5)$$

where $\sigma \in (0, 1)$. It makes sense to enforce this condition only if the model predicts a decrease in the objective function. Thus, filter methods use the switching condition

$$\Delta\phi^{(k)}(\alpha d_x^{(k)}) \geq \delta\eta(x^{(k)})^2,$$

where $\delta > 0$, to decide when (5) should be enforced. If the trial point is accepted, it is added to $\mathcal{F}^{(k)}$ if $\eta(x^{(k)}) > 0$ or if the switching condition fails (which automatically satisfies $\eta(x^{(k)}) > 0$).

4.2 Constraint relaxation strategies

Subproblem constraint relaxation strategies, referred to as constraint relaxation strategies for the sake of conciseness, guarantee that the subproblems are well defined and feasible. In general, we cannot assume that the nonlinear problem (NLP) is locally or globally feasible, nor that a given subproblem is feasible (the linearized constraints may be inconsistent or the trust-region radius may be too small). Moreover, nonlinear solvers may converge to points that violate standard constraint qualifications, and we must take these situations into account when defining optimality conditions. To address these unfavorable scenarios, constraint relaxation strategies relax the constraints systematically or when the standard subproblem is infeasible, thus giving more emphasis to decreasing the constraint violation. We review two constraint relaxation strategies: ℓ_1 relaxation and feasibility restoration.

4.2.1 ℓ_1 relaxation

The ℓ_1 relaxation strategy replaces a constrained optimization problem with a nonsmooth bound-constrained problem in which a penalty term is added to the objective:

$$\begin{aligned} \min_x \quad & \rho f(x) + \|c(x)\|_1 \\ \text{s.t.} \quad & x \geq 0, \end{aligned} \tag{6}$$

where $\rho \geq 0$ is an inverse penalty parameter. An appropriate value of ρ may be determined by steering rules [6, 9, 10].

Note that the penalty parameter is often attached to $\|c(x)\|_1$ in the literature. We adopt the inverse notation as in [9] for several reasons: (i) it is numerically easier to drive ρ to 0 than to drive the penalty parameter of $\|c(x)\|_1$ to $+\infty$; (ii) in second-order methods, the inverse penalty parameter enters the Hessian as objective multiplier; and (iii) as $\rho \rightarrow 0$, only ω_ρ vanishes, and the implicit constraints in ξ (barrier or proximal terms) are still enforced.

The nonsmooth ℓ_1 relaxed problem can be reformulated as a smooth constrained problem with elastic variables. Other norms can be used; however, the ℓ_1 norm has emerged as the preferred option. In particular, the ℓ_1 relaxation is exact: one can show under mild conditions that for $\rho > 0$ sufficiently small, a second-order sufficient point of (6) is also a second-order sufficient point of (NLP) (see, e.g., Theorem 14.3.1 in [18]).

4.2.2 Feasibility restoration

An infeasible subproblem results from inconsistent linearized or bound constraints, or when the trust-region radius is too small; it is an indication that (NLP) may be infeasible. In this case, the method reverts to the *feasibility restoration phase*: the original objective is temporarily discarded, and the following feasibility problem is solved instead:

$$\begin{aligned} \min_x & \|c(x)\| \\ \text{s.t. } & x \geq 0, \end{aligned} \quad (7)$$

for some norm $\|\cdot\|$ in $\mathbb{R}^m$. The aim of solving the feasibility problem is to compute a point as close as possible to the feasible region. Feasibility restoration improves feasibility until a minimum of the constraint violation is obtained or the subproblem becomes feasible again, in which case solving the original problem (the *optimality phase*) is resumed. Any (local) solution $x^* \geq 0$ of (7) with $\|c(x^*)\| > 0$ is a certificate that (NLP) is (locally) infeasible.

4.3 Inequality handling methods

Inequality handling methods deal with the combinatorics induced by inequality constraints within the context of Lagrange-Newton approximations. We briefly review three classes of methods: inequality-constrained methods, equality-constrained methods, and interior-point methods. Note that a classification into inequality-constrained and equality-constrained SQP methods can be found in [39].

4.3.1 Inequality-constrained methods

In inequality-constrained methods, the handling of inequality constraints is deferred to the subproblem solver. Traditionally, sequential quadratic programming (SQP) methods [29, 42, 43, 50] generate a sequence of inequality-constrained quadratic problems (QPs) that are solved by means of an active-set QP solver: it maintains an estimate of the active set and solves a sequence of equality-constrained subproblems. The estimate of the active set is updated at each iteration using dual information, until the algorithm terminates with primal-dual feasibility. SQP methods converge quadratically under reasonable assumptions near a local minimizer, once the active set settles down. Sequential linear programming [11] is a particular case of SQP in which the subproblems are linear problems (LPs); that is, no second-order information is exploited ($H^{(k)} = 0$).

4.3.2 Equality-constrained methods

Equality-constrained methods operate in two phases: the first phase solves a “low-fidelity” subproblem (such as an LP or a convex QP with a quasi-Newton Hessian), which provides an estimate of the active set $\mathcal{A}$. The second phase solves a “high-fidelity” (such as second-order) equality-constrained problem with exact Hessian in which the inequality constraints of the active set are fixed to their active bounds and

the inactive inequalities are dropped. This is illustrated in the following equality-constrained problem (using the notation of Problem (1)):

$$\begin{aligned} \min_x \quad & f(x) \\ \text{s.t.} \quad & \begin{Bmatrix} c(x) \\ Ax \\ x \end{Bmatrix}_{\mathcal{A}} = b_{\mathcal{A}}, \end{aligned}$$

where each component of $b_{\mathcal{A}}$ is the active (lower or upper) bound of the corresponding constraint. A typical example of equality-constrained methods is SLPEQP [11] (aka SLQP): it estimates the active set by solving an LP, then solves an equality-constrained QP (EQP).

4.3.3 Interior-point methods

Primal-dual interior-point methods relax the complementarity equations (2d) by a barrier parameter $\mu > 0$:

$$x_i z_i = \mu, \quad \forall i \in \{1, \dots, n\},$$

which implies $x > 0$ and $z > 0$. Positivity of x and z is then enforced at each iteration, e.g., by a fraction-to-the-boundary rule [49]. Similarly to homotopy methods, μ is asymptotically driven to 0 until the termination criteria are met.

In contrast, *primal interior-point methods* (or barrier methods) treat z as dependent variables. This corresponds to solving the following nonlinear equality-constrained problem:

$$\begin{aligned} \min_x \quad & f(x) - \mu \sum_{i=1}^n \log(x_i) \\ \text{s.t.} \quad & c(x) = 0. \end{aligned}$$

4.4 Hessian models

The Hessian model is a high- or low-quality representation of the Lagrangian Hessian of the original model. It may be:

- the exact Hessian $H^{(k)} = \nabla_{xx}^2 \mathcal{L}_\rho(x^{(k)}, y^{(k)})$ if it is provided by the modeler or the modeling framework;
- a quasi-Newton approximation such as BFGS or SR1, or a limited-memory version thereof;
- the identity matrix;
- the zero matrix.

As the model gets reformulated by the constraint relaxation strategy and the inequality handling method, structured (typically diagonal) curvature may be introduced in the form of a proximal term or a barrier term.

4.5 Inertia correction strategies

Since line-search methods require that $d_x^{(k)}$ be a descent direction, the Lagrangian Hessian or the augmented system is usually regularized such that the former becomes positive definite, and the latter has n positive, m negative, and 0 zero eigenvalues, that is the inertia $(n, m, 0)$. An approximation of the inertia is provided by factorization codes such as MA57 [15].

Typically, the inertia of the Hessian $H^{(k)}$ is corrected by adding a multiple of the identity matrix:

$$H^{(k)} + \delta I$$

for some $\delta > \min\{\text{eig}(H^{(k)})\}$. The inertia of an augmented matrix

$$M \stackrel{\text{def}}{=} \begin{pmatrix} A & B \\ B^T & \end{pmatrix} \quad (8)$$

is controlled by adjusting the regularization parameters $\delta_w \geq 0$ and $\delta_c \geq 0$ in the matrix:

$$\begin{pmatrix} A + \delta_w I & B \\ B^T & -\delta_c I \end{pmatrix}. \quad (9)$$

4.6 Globalization mechanisms

When the methods are started far from a solution, directions may be unbounded or result in trial iterates that increase both the objective and the constraint violation. Globalization mechanisms provide a recourse action if a local approximation is deemed too poor to make progress toward a solution: line-search methods restrict the length of the step along a given direction, while trust-region methods restrict the length of the direction a priori.

4.6.1 Line-search methods

Line-search methods compute a trial step $\alpha d_x^{(k)}$ by determining a step length $\alpha \in (0, 1]$ along the direction $d_x^{(k)}$. Inexact line-search methods determine an approximate step length such that the corresponding trial iterate is accepted by the globalization strategy (sufficient decrease for a merit function or acceptance by a filter). For instance, backtracking line-search methods generate a sequence of trial iterates $x^{(k)} + \alpha^{(k,l)} d_x^{(k)}$ for $l \in \mathbb{N}$ until acceptance, where $\alpha^{(k,l)} = c^l \alpha_{\max}^{(k)}$, $\alpha_{\max}^{(k)} \in (0, 1]$ (usually 1 in SQP methods and smaller than 1 in IPM) and $c \in (0, 1)$.

Line-search methods require that the approximation $H^{(k)}$ of the Lagrangian Hessian be positive definite on the nullspace of the Jacobian of the active constraints in order to guarantee that $d_x^{(k)}$ is a descent direction for the merit function.

4.6.2 Trust-region methods

Trust-region methods limit the length of the direction d_x a priori by imposing the trust-region constraint $\|d_x\| \leq \Delta^{(l)}$ for some norm $\|\cdot\|$ in $\mathbb{R}^n$, where $\Delta^{(l)} > 0$ is the trust-region radius. The step $d_x^{(k,l)}$ is then computed by (approximately) solving the trust-region subproblem. If the trial iterate $\hat{x}^{(k+1,l)} \stackrel{\text{def}}{=} x^{(k)} + d_x^{(k,l)}$ is accepted by the globalization strategy, $\Delta^{(l)}$ is increased if the trust region is active at $d_x^{(k,l)}$. Otherwise, $\Delta^{(l)}$ is decreased to a value smaller than $\min(\Delta^{(l)}, \|d_x^{(k,l)}\|)$. Contrary to line-search methods, a positive definite Lagrangian Hessian is not required because directions of negative curvature are bounded by the trust region.

4.7 Summary

Table 6 presents a unified view of state-of-the-art solvers ALGLIB MinNLC [4], CONOPT [14], FICO XSLP [2], filterSQP [20], IPOPT [49], KNITRO [8], LANCELOT [12], LOQO [46], MINOS [38], NAG e04uc/e04wdc [1], NLPQL [44], SLSQP [31], SNOPT [27], SQuID [6], and WORHP [7]. Each solver is characterized in terms of the ingredients within the proposed unifying framework. Note that FICO XSLP is the only solver that does not implement a proper globalization strategy.

5 Uno: a unified solver for nonlinearly constrained optimization

We have implemented our unifying framework for nonlinearly constrained optimization within Uno, a modular solver written in C++17. A generic and flexible code, it supports a broad range of strategies that can be combined automatically and on the fly with no programming effort from the user. Additional software components are implemented orthogonally to the eight ingredients, such as termination criteria. The code is packaged in a lightweight library (around 10,000 lines of code for the current version Uno 2.2.0, excluding interfaces) available as open-source software under the MIT license at <https://github.com/cvanaret/Uno>. Uno 2.2.0 is available via its C, Julia (registered package UnoSolver.jl), Python (the packaging of unopy is under development), Fortran, and AMPL [24, 26] interfaces.

In the following, we briefly present the generic architecture of Uno 2.2.0, describe how ingredients can be automatically combined, list its features, and discuss the current limitations.

5.1 Generic architecture

The modularity of Uno stems from its generic architecture: each ingredient is implemented once and independently of the others, which improves readability, and makes building blocks less prone to error and easier to maintain than monolithic codes. This results in a modern, flexible, and maintainable framework.

Figure 5 represents Uno's object-oriented architecture as a Unified Modeling Language (UML)² diagram based on inheritance ("is a") and composition ("has a"). The ingredients are modeled as abstract classes: they define interfaces, that is, generic actions and behaviors that must be implemented by concrete strategies modeled as subclasses. For example:

- the classes `BacktrackingLineSearch` and `TrustRegionStrategy` inherit from the abstract class `GlobalizationMechanism` (inheritance is represented by dashed arrows) and thus must implement the purely virtual member function `compute_acceptable_iterate()`;
- the `GlobalizationMechanism` class possesses a member of type `ConstraintRelaxationStrategy` (composition is represented by solid diamond lines).

5.2 Automatic strategy combinations

Uno 2.2.0 implements state-of-the-art strategies that can be combined automatically thanks to the modular software architecture. The number of possible strategy combinations is the size of the Cartesian product of the eight ingredients. Note that all combinations do not necessarily result in sensible algorithms, or even convergent approaches.

At the moment, Uno prohibits the combination of `interior-point methods` and `trust-region strategies`. A possible strategy is KNITRO's step decomposition: the direction is decomposed into a normal step that minimizes the constraint violation within the trust region, and a tangential step that minimizes the objective for a given constraint violation target within the trust region. This limitation will be resolved in later Uno versions.

Some strategy combinations that correspond to state-of-the-art solvers are available as "presets": they automatically connect the eight ingredients, and set values for the hyperparameters that can be found in the solvers' documentations (they will be listed in the Uno user manual). The following presets are available in Uno 2.2.0:

- `filtersqp`: A trust-region restoration filter SQP method à la `filterSQP` [19]. Second-order correction steps were not implemented.
- `ipopt`: A line-search restoration filter interior-point method à la `IPOPT` [49]. Second-order correction steps, scaling, least-square multipliers, iterative refinement, iterative bound relaxations, non-monotone techniques, and soft feasibility restoration were not implemented.

5.3 Features of Uno 2.2.0

Uno 2.2.0 contains a number of features that are described here.

² <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/>.

5.3.1 Interfaces to subproblem solvers

Interfaces to the following subproblem solvers are available:

- BQPD [17, 23]: a null-space active-set solver for nonconvex QPs. BQPD accepts Hessian-vector products instead of an explicit matrix. A Hessian linear operator can be provided to the Uno model;
- MA57 [15], MA27 [16], and MUMPS [3]: direct solvers for sparse symmetric indefinite linear systems;
- HiGHS [30]: a parallel simplex implementation for linear programming. Uno currently does not support the HiGHS QP solvers.

We plan to write an interface to GALAHAD's TRS package [25] that solves the trust-region problem to global optimality:

$$\begin{aligned} \min_d \quad & \frac{1}{2}d^T H d + c^T d + f \\ \text{s.t.} \quad & A d = 0 \\ & \|d\|_2 \leq \Delta. \end{aligned}$$

5.3.2 Definition of the subproblem

The constraint relaxation strategy and the inequality handling method successively reformulate the original problem with respect to the general constraints $c(x) = 0$ and the bound constraints $x \geq 0$. A local model of the resulting reformulated problem is then built.

The Lagrange-Newton subproblem is composed of the following elements: the reformulated problem, the current primal-dual iterate, the Hessian model, the inertia correction strategy, and a possible trust-region radius. These elements interact with one another to automatically define the progress measures and their local models (see Section 5.3.3), the Lagrangian Hessian or augmented matrix with the correct inertia (see Section 5.3.4), and the type of subproblem solvers that can be used (see Section 5.3.5).

The subproblem is represented schematically in Figure 3. Dashed arrows represent zeroth- and first-order calls. Full arrows represent calls to the Lagrangian Hessian. Dotted arrows represent calls to the inertia correction strategy for a given Hessian $H^{(k)}$ or augmented system $M^{(k)}$.

5.3.3 Definition of progress measures

The progress measures (see Section 4.1) are defined automatically by the successive reformulations of the optimization problem (11RelaxedProblem and PrimalDualInteriorPointProblem):

- the norm of the infeasibility measure is determined by that of the feasibility problem or that of the relaxed problem);

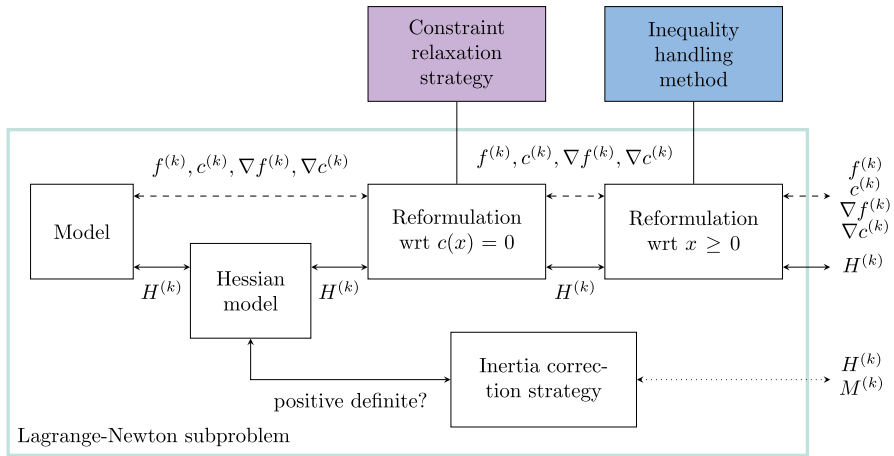


Fig. 3 Definition of a Subproblem

- the objective measure is $\omega_\pi(x) \stackrel{\text{def}}{=} \pi f(x)$ (for all the currently implemented strategies);
- the auxiliary measure accumulates proximal (in the feasibility problem 11Relaxed Problem) and barrier (in Primal Dual Interior Point Problem) terms.

The fact that the progress measures (respectively their local models) are closely linked to the definition of the problem (respectively of the subproblem) guarantees that, as much as possible, a given combination of strategies is meaningful. This assumes that i) the approximate solution to the subproblem satisfies a sufficient decrease condition (e.g., it achieves a fraction of the Cauchy decrease) or that the direction is a descent direction for the KKT residuals, and ii) that the reformulations are (asymptotically) equivalent to the original problem.

5.3.4 Inertia correction

Each Hessian model (see Section 4.4) declares whether it is positive definite (the exact Hessian is “indefinite” by default), and whether it is available as an explicit matrix and as a linear operator. The inertia of the Lagrangian Hessian is corrected if:

- the Hessian model is not already positive definite; and
- the Hessian model is available as an explicit matrix (otherwise, Uno terminates with an error message); and
- the inertia correction strategy performs primal correction (“primal” or “primal-dual”).

Furthermore, the successive reformulations (11RelaxedProblem and PrimalDualInteriorPointProblem) maintain the lists of variables and constraints that should be primally and dually regularized, respectively. The expected inertia is computed accordingly. For instance, 11RelaxedProblem adds elastic variables that enter the objective and constraints only linearly. Consequently, they contribute to the number of zero eigenvalues.

5.3.5 Instantiation of ingredients

Most ingredients are picked by the user via options. The following ingredients may be set or overridden by Uno: after analyzing the

- for inequality-constrained methods, the subproblem solver is picked depending whether the subproblems have curvature. If this is the case, a QP solver is allocated. Otherwise, an LP solver is allocated;
- if the reformulated problem is unconstrained,
 - the constraint relaxation strategy is set to the custom `NoRelaxation` strategy;
 - the globalization strategy is set to `l1MeritFunction`;
 - and there is no curvature in the subproblem, the subproblem solver is set to the custom `BoxLPSolver`;
- if the Hessian model is “exact” but neither an explicit Hessian matrix nor a Hessian linear operator was provided by the modeler, the Hessian model is set to “zero” (as long as quasi-Newton approximations are not implemented in Uno) and a warning message is printed.

5.3.6 Sparse matrix formats

Each subproblem solver possesses an object that inherits from the abstract class `EvaluationSpace` in which they store the Jacobian matrix, the Hessian matrix, or the augmented matrix in specific sparse formats:

- BQPD expects a Jacobian in Compressed Sparse Row (CSR) format;
- MA57, MA27, and MUMPS expect a matrix in COOrdinate (COO) format;
- HiGHS expects the Jacobian in Compressed Sparse (Row or Column) format.

To avoid redundancy, these matrices exist in a single sparse format and at a single location in the code. The `EvaluationSpace` object is passed around throughout the framework and provides Hessian-vector, Jacobian-vector, and Jacobian-transposed-vector products (used for instance for computing the stationarity residual).

5.3.7 Termination criteria

In practice, we cannot hope to drive the error in the first-order conditions to zero: optimization algorithms may terminate at locally infeasible points or at points that fail to satisfy constraint qualifications. We therefore check for termination, rather than for optimality. Uno terminates at the primal-dual iterate (x^*, y^*, z^*, π^*) if

- sufficient first-order optimality conditions are approximately satisfied:
 - a **feasible KKT point** (CQ holds) if it satisfies (2a), (2b), (2c), and (2d) with $\pi^* > 0$;
 - a **feasible FJ point** (CQ does not hold) if it satisfies (2a), (2b), (2c), and (2d) with $\pi^* = 0$;

- an **infeasible stationary point** (a minimum of the constraint violation) if it satisfies (2a) and (2c) with $\pi^* = 0$, no primal feasibility $c(x^*) > 0$ and a complementarity condition on the violated constraints that depends on the norm used in the feasibility problem;
- primal feasibility is approximately satisfied and the trust-region radius is close to machine epsilon. This may occur when the problem is non-differentiable, when the gradients are erroneous, or when second-order derivatives are not used, as is the case in Sequential Linear Programming (SLP) methods;
- the first-order optimality conditions cannot be satisfied for the user-defined tolerance ε but are satisfied for a looser tolerance (e.g., 100ε) for a certain number of consecutive iterations (e.g., 15).

5.3.8 Error handling

Uno terminates with an error message if it encounters an IEEE exception at the initial point $x^{(0)}$. Otherwise, it tries to recover from IEEE exceptions during the optimization process by invoking the globalization mechanism in the following way:

- the current trust-region radius is reduced;
- the backtracking line search reduces the tentative step length.

6 Combining the ingredients

We show below how the eight ingredients naturally arise in a popular trust-region restoration filter SQP method and a line-search restoration filter interior-point method. We also provide complete multicolor pseudocodes in Algorithms 2 and 3. They highlight the complexity of algorithm design, and the contrast between the basic (simplistic) abstract algorithm (Algorithm 1) and a robust and convergent optimization algorithm. In addition, the images illustrate the challenges we faced when implementing Uno.

6.1 Trust-region restoration filter SQP

Convergence of SQP filter methods has been proven under mild conditions in [19, 22] in the context of trust-region methods and in [47, 48] in the context of line-search methods. The trust-region optimality QP subproblem about $(x^{(k)}, y^{(k)})$ is defined as

$$\begin{aligned}
 \min_{d_x} \quad & \frac{1}{2} d_x^T H^{(k)} d_x + 1 (\nabla f^{(k)})^T d_x \\
 \text{s.t.} \quad & c^{(k)} + (\nabla c^{(k)})^T d_x = 0 \\
 & x^{(k)} + d_x \geq 0 \\
 & \|d_x\|_\infty \leq \Delta^{(l)},
 \end{aligned} \quad (QP^{(k)}(\Delta^{(l)}))$$

where $\Delta^{(l)} > 0$ is the current trust-region radius. If $(QP^{(k)}(\Delta^{(l)}))$ is infeasible ($\Delta^{(l)}$ is too small or the linearized constraints are inconsistent), we switch to feasibility restoration and solve a smooth reformulation of the ℓ_1 feasibility problem with elastic variables $u^+ \in \mathbb{R}^m$ and $u^- \in \mathbb{R}^m$:

$$\begin{aligned} \min_{d_x, u^+, u^-} \quad & \frac{1}{2} d_x^T H^{(k)} d_x + e^T u^+ + e^T u^- \\ \text{s.t.} \quad & c^{(k)} + (\nabla c^{(k)})^T d_x - u^+ + u^- = 0 \\ & x^{(k)} + d_x \geq 0 \\ & \|d_x\|_\infty \leq \Delta^{(l)} \\ & u^+ \geq 0, u^- \geq 0. \end{aligned} \quad (FQP^{(k)}(\Delta^{(l)}))$$

If the trial iterate $x^{(k)} + d_x$ makes sufficient progress with respect to the filter method, it is accepted and we start a new iteration; if the trust region was active at the solution of the QP ($\|d_x^*\|_\infty = \Delta^{(l)}$), we enlarge the radius. If the trial iterate is rejected, we resolve the trust-region subproblem with a smaller trust-region radius. It can be shown that this mechanism either generates an acceptable iterate or results in an infeasible QP. The complete pseudocode of the method is given in Algorithm 2.

Note that in [19] the authors do not solve the feasibility problem with elastic variables but exploit the partition into satisfied and violated linearized constraints provided by the Phase I method of the QP solver BQPD.

6.2 Line-search filter restoration infeasible interior-point method

An infeasible interior-point method does not require feasibility with respect to the general constraints. A prerequisite is to turn inequality constraints into equality constraints using slack variables:

$$\begin{aligned} \min_x f(x) \\ \text{s.t.} \quad & l_c \leq c(x) \leq u_c \\ & l_x \leq x \leq u_x \end{aligned} \quad \Longleftrightarrow \quad \begin{aligned} \min_{x,s} f(x) \\ \text{s.t.} \quad & c(x) - s = 0 \\ & l_x \leq x \leq u_x \\ & l_c \leq s \leq u_c, \end{aligned}$$

before applying the reformulation described in Section 4.3.3.

Provided that the subproblem is convex, the primal-dual direction is the solution of a Newton linear system, the *primal-dual system* (or full-space system), in which the multipliers z are treated as independent variables. We solve a smaller, symmetrized

version of the primal-dual system:

$$\begin{pmatrix} H^{(k)} + (X^{(k)})^{-1}Z^{(k)} + \delta_w I & \nabla c^{(k)} \\ \text{1} & \\ (\nabla c^{(k)})^T & -\delta_c I \end{pmatrix} \begin{pmatrix} d_x \\ -d_y \end{pmatrix} = \begin{pmatrix} \text{1} \nabla f^{(k)} - \nabla c^{(k)} y^{(k)} - \mu (X^{(k)})^{-1} e \\ c^{(k)} \end{pmatrix}, \quad (IPSP_\mu)$$

where $X^{(k)} = \text{diag}(x^{(k)})$, $Z^{(k)} = \text{diag}(z^{(k)})$, e is a vector of ones of appropriate size, δ_w and δ_c are primal and dual inertia correction coefficients, respectively. The dual direction for the bound constraints is given by $d_z = (X^{(k)})^{-1}(\mu e - Z^{(k)}d_x) - z^{(k)}$. The fraction-to-boundary rule determines primal and dual step lengths that maintain positivity of x and z :

$$\begin{aligned} \alpha_x^{(k)} &\stackrel{\text{def}}{=} \max\{\alpha \in (0, 1] \mid x^{(k)} + \alpha d_x \geq (1 - \tau)x^{(k)}\} \\ \alpha_z^{(k)} &\stackrel{\text{def}}{=} \max\{\alpha \in (0, 1] \mid z^{(k)} + \alpha d_z \geq (1 - \tau)z^{(k)}\}, \end{aligned} \quad (10)$$

where τ is a parameter close to 1. A filter line search assesses whether the trial iterate makes sufficient progress with respect to the filter method. If so, the trial iterate is accepted; otherwise the line search backtracks and tries again with a smaller step length. If the step length ultimately falls below a given threshold (e.g., 10^{-7}), we switch to feasibility restoration and solve the ℓ_1 feasibility problem with elastic variables.

Note that by construction the filter entries depend on the barrier parameter μ through the auxiliary measure ξ . Consequently, the filter must be flushed whenever μ is updated. The complete pseudocode of the method is given in Algorithm 3. An alternative that we plan to explore in the future is to update the filter entries whenever the barrier parameter is updated.

7 Numerical results

We now compare the filtersqp and ipopt presets of Uno 2.2.0 against state-of-the-art solvers filterSQP (20010817, with the QP solver BQPD), IPOPT 3.14.20 (with the linear solver MA57 3.10.0), SNOPT 7.5-1.2, MINOS 5.51, LANCELOT, LOQO 7.03, and CONOPT 3.17A on 429 small test problems of the CUTE benchmark [5] translated into AMPL³. Their dimensions are reported in Table 1. The full name of nuffield is nuffield_continuum, which the table is too narrow to contain. The log files of all the solvers are available in the following repository:

https://github.com/cvanaret/nonlinear_optimization_solver_benchmark. IPOPT terminated with the status “EXIT: Problem has too few degrees of freedom” on the instances argauss and lewispol; this is interpreted as failure.

³ The AMPL translations are available at https://arnold-neumaier.at/glopt/coconut/Benchmark/Library2_new_v1.html

Table 1 Dimensions of the 429 small CUTE instances.

Instance	n	m	Instance	n	m	Instance	n	m	Instance	n	m	Instance	n	m
3pk	30	0	dual1	85	1	hs021	2	1	hs110	10	0	palmer3	4	0
aircfta	5	5	dual2	96	1	hs022	2	2	hs111inp	10	3	palmer4a	6	0
aircftb	5	0	dual4	75	1	hs023	2	5	hs111	10	3	palmer4b	4	0
airport	84	42	dualc1	9	13	hs024	2	2	hs112	10	3	palmer4c	8	0
aljazaf	3	1	dualc2	7	9	hs025	3	0	hs113	10	8	palmer4e	8	0
allinitc	3	1	dualc5	8	1	hs026	3	1	hs114	10	11	palmer4	4	0
allinit	3	0	dualc8	8	15	hs027	3	1	hs116	13	15	palmer5a	8	0
allinitu	4	0	eg1	3	0	hs028	3	1	hs117	15	5	palmer5b	9	0
alsotame	2	1	eigencco	30	15	hs029	3	1	hs118	15	17	palmer5c	6	0
argauss	3	15	eigmaxc	22	22	hs030	3	1	hs119	16	8	palmer5d	4	0
arglinb	10	0	eigminc	22	22	hs031	3	1	hs21mod	7	1	palmer5e	8	0
argline	8	0	engval2	3	0	hs032	3	2	hs268	5	5	palmer6a	6	0
avgas	8	10	errinros	50	0	hs033	3	2	hs35mod	2	1	palmer6c	8	0
avgasb	8	10	expfta	5	21	hs034	3	2	hs3mod	2	0	palmer6e	8	0
avion2	49	15	expfit	2	0	hs035	3	1	hs44new	4	5	palmer7a	6	0
bard	3	0	extrasm	2	1	hs036	3	1	hs99exp	28	21	palmer7c	8	0
batch	46	69	extrosnb	10	0	hs037	3	1	hubfit	2	1	palmer7e	8	0
beale	2	0	fecu	19	8	hs038	4	0	humps	2	0	palmer8a	6	0
biggs3	3	0	fletcher	4	4	hs039	4	2	hypcir	2	2	palmer8c	8	0
biggs5	5	0	genhs28	10	8	hs040	4	3	jensmp	2	0	palmer8e	8	0
biggs6	6	0	genhumps	5	0	hs041	4	1	kiwresc	3	2	pentagon	6	12
biggsc4	4	7	gigomez1	3	3	hs042	3	1	kowosb	4	0	pfit1ls	3	0
booth	2	2	goffin	51	50	hs043	4	3	lakes	90	78	pfit1	3	0
box2	2	0	gotfr	2	2	hs044	4	6	launch	25	29	pfit2ls	3	0
box3	3	0	gridnetg	44	34	hs045	5	0	lewispol	6	9	pfit2	3	0

Table 1 continued

Instance	<i>n</i>	<i>m</i>	Instance	<i>n</i>	<i>m</i>	Instance	<i>n</i>	<i>m</i>	Instance	<i>n</i>	<i>m</i>
bqp1var	1	0	gridneth	61	36	hs046	5	2	linspanh	72	32
bpggabim	46	0	gridneti	61	36	hs047	5	3	loadbal	31	31
bpggasim	50	0	growths	3	0	hs048	5	2	loghairy	2	0
brkmcc	2	0	growth	3	0	hs049	5	2	logros	2	0
browal	10	0	gulf	3	0	hs050	5	3	lootsma	3	2
browbns	2	0	hadamals	90	0	hs051	5	3	lotschd	12	7
brownden	4	0	haifas	7	9	hs052	5	3	lsnmodoc	5	4
bt10	2	2	haity	2	0	hs053	5	3	lsqfit	2	1
bt11	5	3	haldmads	6	42	hs054	6	1	madsen	3	6
bt12	5	3	harr6	6	0	hs055	6	6	makela1	3	2
bt13	5	1	hatfda	4	0	hs056	7	4	makela2	3	3
bt1	2	1	hatfdb	4	0	hs057	2	1	makela3	21	20
bt2	3	1	hatfdc	4	0	hs059	2	3	makela4	21	40
bt3	5	3	hatfdd	3	0	hs060	3	1	maratosb	2	0
bt4	3	2	hatfde	3	0	hs061	3	2	maratos	2	1
bt5	3	2	hatfdf	3	3	hs062	3	1	matrix2	6	2
bt6	5	2	hatfdg	25	25	hs063	3	2	maxlika	8	0
bt7	5	3	hatfdh	4	7	hs064	3	1	mconcon	15	11
bt8	5	2	heart6ls	6	0	hs065	3	1	mdhole	2	0
bt9	4	2	heart6	6	6	hs066	3	2	methanb8	31	0
byrdsphr	3	2	heart8ls	8	0	hs067	10	7	methanl8	31	0
camel6	2	0	heart8	8	8	hs070	4	1	mexhat	2	0
cantilvr	5	1	helix	3	0	hs071	4	2	meyer3	3	0
catena	32	11	hiliberta	10	0	hs072	4	2	mifflin1	3	2
cb2	3	3	hilbertb	50	0	hs073	4	3	mifflin2	3	2
cb3	3	3	himmelba	2	2	hs074	4	4	minmaxbd	5	20
									rosenbr	2	0

Table 1 continued

Instance	n	m	Instance	n	m	Instance	n	m	Instance	n	m	Instance	n	m
chaconn1	3	3	himmelbb	2	0	hs075	4	4	minmaxrb	3	4	rosenmmx	5	4
chaconn2	3	3	himmelbc	2	2	hs076	4	3	minsurf	36	0	s365mod	7	5
chebyqad	50	0	himmelbd	2	2	hs077	5	2	mistake	9	13	sim2bqp	2	0
chnrosnb	50	0	himmelbe	3	3	hs078	5	3	model	60	32	simbqp	2	0
cliff	2	0	himmelbf	4	0	hs079	5	3	mwright	5	3	simplipa	2	2
cluser	2	2	himmelbg	2	0	hs080	5	3	nasty	2	0	simpllpb	2	3
concon	15	11	himmelbh	2	0	hs081	5	3	nonmsqrt	9	0	sineali	20	0
congimgz	3	5	himmelbk	24	14	hs083	5	3	nuffield	2	1	sineval	2	0
coolhans	9	9	himmelpl	2	0	hs084	5	3	obstclal	64	0	sisser	2	0
core1	65	50	himmelpl2	2	1	hs085	5	36	obstclbl	64	0	snake	2	2
coshtun	61	20	himmelpl3	2	2	hs086	5	6	obstclbu	64	0	spanhyd	72	32
cresec4	6	8	himmelpl4	2	3	hs087	9	4	odfits	10	6	spiral	3	2
csfi1	5	4	himmelpl5	2	3	hs088	2	1	optcntrl	28	20	ssnlbeam	31	20
csfi2	5	4	himmelpl6	2	4	hs089	3	1	optmass	66	55	stancmin	3	2
cube	2	0	hong	4	1	hs090	4	1	optprloc	30	29	supersim	2	2
dallass	44	29	hs001	2	0	hs091	5	1	orthregb	27	6	swopf	82	91
deconvb	51	0	hs002	2	0	hs092	6	1	orthrega	36	20	synthesl	6	6
deconvc	51	1	hs003	2	0	hs093	6	2	osbornea	5	0	tame	2	1
deconvu	51	0	hs004	2	0	hs095	6	4	osborneb	11	0	tointqor	50	0
degenlpa	20	14	hs005	2	0	hs096	6	4	oslbqp	8	0	try-b	2	1
degenlpb	20	15	hs006	2	1	hs097	6	4	palmerla	6	0	twobars	2	2
demymalo	3	3	hs007	2	1	hs098	6	4	palmerlb	4	0	vanderbm4	9	17
denschna	2	0	hs008	2	2	hs099	19	14	palmerlc	8	0	watson	31	0
denschbn	2	0	hs009	2	1	hs100lhp	7	2	palmerld	7	0	weeds	3	0

[illegible]

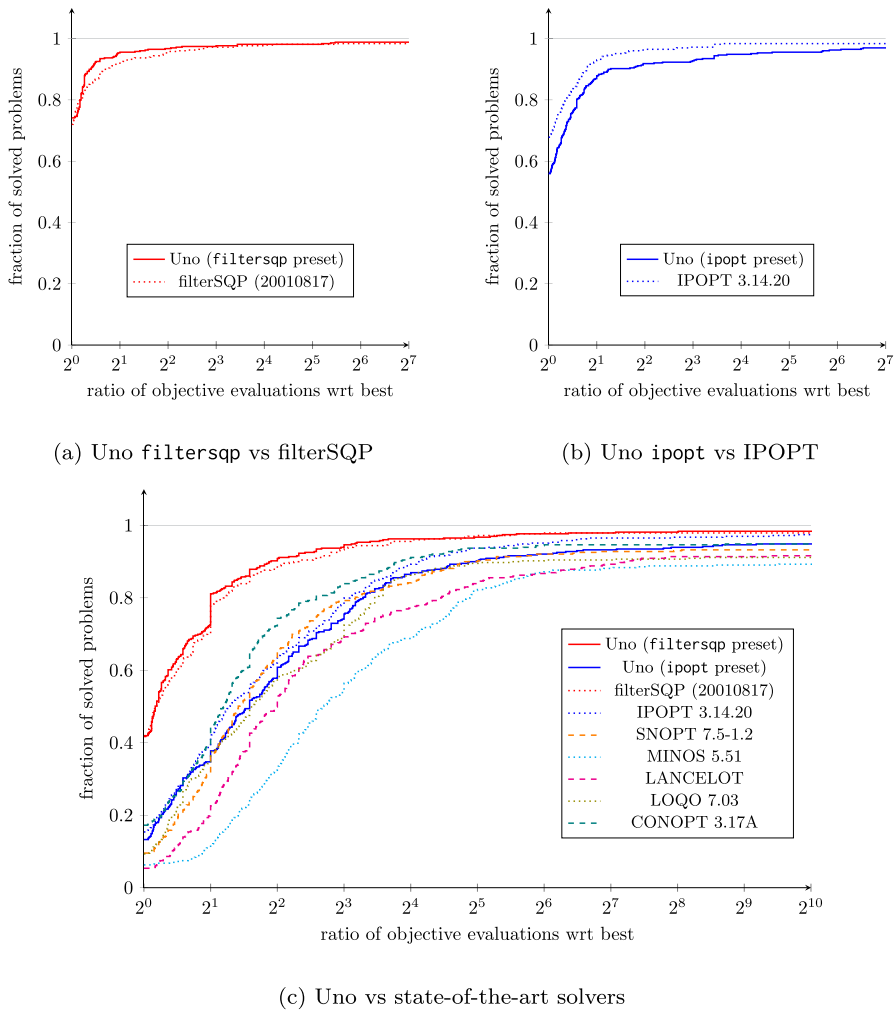


Fig. 4 Performance profile on 429 small CUTE problems: number of objective evaluations

In this section we demonstrate that the Uno presets closely mimic the corresponding state-of-the-art solvers filterSQP and IPOPT. Figures 4a and 4b portray performance profiles [13] of the filtersqp preset vs filterSQP, and of the ipopt preset vs IPOPT, respectively. Figure 4c illustrates the overall picture.

On all three plots, the y axis is the fraction of solved problems, and the x axis represents the relative budget of objective evaluations compared with the (virtual) best solver for each instance. The higher and more to the left, the better. These performance profiles demonstrate that the Uno presets filtersqp and ipopt mimic the corresponding state-of-the-art solvers well and perform well with respect to all the state-of-the-art solvers. These results validate the implementation of off-the-shelf strategies within Uno.

Table 2 Shifted geometric mean ($s = 10$) of objective evaluations on 429 small CUTE instances.

Uno presets		State-of-the-art solvers						
filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
18.08	48.45	18.73	34.63	61.02	159.78	92.27	71.44	43.15

Table 3 Number of objective evaluations on a subset of CUTE LPs.

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	FilterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
booth	2	2	2	2	2	1	3	7	1
degenlp	2	44	2	29	26	15	25	29	1
degenlpb	2	42	2	41	26	23	45	30	1
extrasim	2	6	2	6	1	1	3	11	1
goffin	2	7	3	8	25	25	10	13	1
himmelba	2	2	2	2	2	1	3	7	1
linspanh	2	18	2	19	5	14	13	12	1
makela4	3	8	3	8	1	1	20	12	1
model	2	22	2	15	23	34	33	15	1
res	2	9	2	10	1	5	1	12	1
simpllp	2	13	2	14	3	3	5	12	1
simpllpb	2	10	2	11	1	1	4	13	1
supersim	2	7	2	2	1	1	7	11	1
zangwil3	2	2	2	2	3	2	3	8	1

Table 3, Table 4, and Table 5 summarize the number of objective evaluations required by each solver for linear, quadratic, and nonlinear instances, respectively. The lowest count is shown in bold. 12 instances (core1, csfi2, discs, growth, hs085, hs093, hs114, launch, meyer3, polak6, spanhyd, vanderm4) are solved by IPOPT but not by the Uno ipopt preset, most likely because of the IPOPT features that were not implemented (see Section 5.2).

Table 2 shows for each solver the shifted geometric mean of its objective evaluations $e_1, \dots, e_N: \left(\prod_{i=1}^N (e_i + s) \right)^{\frac{1}{N}} - s$, where $s = 10$ is the shift factor, and $N = 429$ is the number of instances. Upon failure on instance i , we take $e_i = 10^6$.

Table 4 Number of objective evaluations on a subset of CUTE QPs.

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
3pk	7	11	7	12	41	209	47	20	24
arglinb	2	3	2	3	1	8	2	3	20
arglinc	2	5	2	3	1	8	2	3	18
avgasa	2	10	2	10	9	19	12	11	16
avgasb	2	13	2	13	9	15	11	16	19
biggsc4	2	26	2	36	13	16	15	21	4
bqp1var	2	6	2	6	1	5	3	10	2
bqpgabim	2	17	2	21	36	107	6	14	17
bqpgasim	2	17	2	21	40	121	6	14	15
bt3	2	2	2	2	5	12	6	3	9
deconvb	42	8514	30	—	103	30	31	45	23
dixon3dq	2	2	2	2	10	43	4	3	9
dual1	2	17	2	17	221	—	9	22	29
dual2	2	14	2	14	116	—	9	19	27
dual4	2	14	2	14	31	—	8	21	23
dualc1	2	30	2	29	7	24	17	26	14
dualc2	2	21	2	27	5	14	19	21	17
dualc5	2	11	2	11	7	24	8	16	15
dualc8	2	15	2	13	9	16	24	19	25
fccu	4	2	4	2	19	48	13	3	11
genhs28	2	2	2	2	11	12	5	3	9
hatfldc	5	6	5	6	14	19	9	9	15
hatfldh	2	14	2	19	3	8	13	16	5
hilberta	2	2	2	2	2	12	60	3	9
hilbertb	2	2	2	2	50	—	5	3	10
hs003	2	6	2	5	2	12	16	11	5
hs021	2	7	2	9	1	8	3	12	12
hs028	2	2	2	2	4	12	4	3	8
hs035	2	8	2	8	5	12	8	10	12
hs044	2	20	2	21	2	10	11	16	5
hs048	2	2	2	2	6	16	3	3	9
hs051	2	2	2	2	6	12	10	3	7
hs052	2	2	2	2	5	11	6	3	9
hs053	2	7	2	7	2	12	6	12	15
hs054	2	8	2	8	5	16	9	12	12
hs076	2	8	2	8	4	13	9	11	16
hs118	3	12	3	12	21	41	19	17	4
hs21mod	2	16	2	17	1	8	3	20	12

Table 4 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
hs268	2	19	2	17	6	36	27	27	19
hs35mod	2	16	2	16	1	6	3	16	6
hs3mod	2	6	2	6	5	13	4	12	10
hs44new	2	14	2	14	4	10	10	21	4
lotschd	2	15	3	15	8	5	9	19	7
lsqfit	2	8	2	8	3	11	7	12	10
maratosb	10	33	13	33	7	10	8	21	14
nasty	2	2	2	2	1	6	5	3	10
obstclal	2	11	2	15	37	90	7	23	15
obstclbl	2	12	2	13	44	87	3	13	47
obstclbu	2	13	2	13	36	74	2	15	26
oslbqp	2	15	2	15	6	11	3	19	2
palmer1c	7	23	7	2	8	61	145	–	–
palmer1d	6	26	6	2	7	50	34	–	–
palmer2c	5	3	5	2	8	62	298	–	–
palmer3c	5	2	5	2	8	61	206	–	–
palmer4c	6	2	6	2	8	61	176	–	–
palmer5c	4	2	4	2	6	26	2	–	–
palmer5d	5	2	5	2	4	25	2	–	–
palmer6c	6	2	6	2	8	60	159	–	–
palmer7c	8	3	8	2	8	64	189	–	–
palmer8c	7	2	7	2	8	61	152	–	–
portfl1	2	10	2	10	12	48	20	17	20
portfl2	2	9	2	9	12	50	14	17	23
portfl3	2	11	2	11	13	52	14	17	32
portfl4	2	10	2	10	11	49	19	16	19
portfl6	2	9	2	9	11	48	20	16	31
qudlin	2	24	2	26	11	18	2	20	3
sim2bqp	2	8	2	8	2	9	3	14	5
simbqp	2	8	2	8	2	10	2	13	4
tame	2	6	2	6	1	8	2	9	6
tointqor	2	2	2	2	50	–	8	3	12
zangwil2	2	2	2	2	2	11	4	3	3
zecevic2	2	9	2	9	2	9	7	11	7

Table 5 Number of objective evaluations on a subset of CUTE NLPs.

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
aircrafta	4	4	4	4	–	1	10	6	2
aircraftb	21	15	21	19	58	65	27	21	32
airport	14	21	13	16	58	527	69	19	101
aljazzaaf	11	79	15	82	145	64	24	79	9
allinitc	29	34	24	44	105	55	76	–	17
allinit	11	15	11	19	17	29	13	18	16
allinitu	12	15	12	15	14	20	15	11	17
alsotame	5	9	5	9	6	11	11	11	4
argauss	1	–	1	–	7	–	–	–	3
avion2	23	6918	19	136	19	17	787	65	25
bard	11	9	11	9	23	36	15	19	17
batch	9	62	9	34	33	379	–	64	32
beale	19	11	10	19	15	24	21	10	16
biggs3	11	16	11	28	24	31	40	12	15
biggs5	54	24	50	36	107	38	64	32	48
biggs6	74	38	83	50	120	119	103	59	139
box2	9	9	9	9	10	10	20	10	10
box3	8	11	8	15	24	15	31	11	13
brkmcc	4	4	4	4	10	13	7	8	10
brownal	8	8	8	8	21	77	24	10	17
brownbs	48	9	48	8	32	35	7	49	1
brownden	9	9	9	9	40	41	9	57	15
bt10	8	9	7	7	1	1	20	11	3
bt11	9	10	7	9	12	56	22	11	21
bt12	5	5	5	5	10	59	11	11	11
bt13	24	26	48	25	34	69	–	21	590
bt1	2	15	1	15	12	16	19	24	1
bt2	13	14	13	13	18	385	36	18	20
bt4	9	12	11	10	10	47	25	9	21
bt5	10	9	9	8	11	172	20	8	10
bt6	11	12	12	18	14	117	25	12	28
bt7	17	117	19	30	36	85	49	18	8
bt8	23	31	12	50	14	21	30	–	9
bt9	19	31	23	14	19	34	22	14	17
byrdsphr	8	98	11	19	59	35	43	11	17
camel6	9	11	8	11	19	24	8	12	19
cantilvr	14	16	16	12	23	69	27	16	45
catena	13	54	13	7	87	125	56	26	80
cb2	8	16	7	9	7	31	18	11	14
cb3	8	15	7	10	1	26	18	11	3

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
chaconn1	6	13	5	7	9	22	12	11	10
chaconn2	6	12	5	7	1	20	11	11	4
chebyqad	64	111	50	139	–	–	62	12	87
chnrosnb	56	55	59	92	170	–	68	58	101
cliff	28	28	28	24	28	46	28	32	32
cluster	10	10	10	10	–	3	45	11	3
concon	3	10	5	10	14	–	676	–	40
congigmz	5	40	4	33	10	16	30	36	3
coolhans	4	10	3	10	–	4	281	25	2
core1	5	–	6	86	44	86	–	52	5
coshfun	86	41226	303	725	273	1091	154	24	140
cresc4	16	5311	52	79	93	864	–	111	47
csfi1	35	28	18	12	36	–	155	16	7
csfi2	11	–	8	86	60	–	180	17	3
cube	39	38	41	58	42	66	52	41	36
dallass	18	36	56	29	109	140	–	–	–
deconvc	24	194	58	99	81	87	43	31	30
deconvu	4033	94	971	729	152	30	69	76	86
demyalo	9	19	8	12	8	34	28	15	15
denschna	7	7	7	7	12	23	13	9	10
denschnb	10	21	10	25	10	17	11	9	10
denschnc	11	11	11	11	21	30	13	14	13
denschnd	41	45	43	27	77	111	65	44	64
denschne	11	12	11	25	44	34	16	15	34
denschnf	7	7	7	7	12	21	8	11	12
dipigri	13	37	13	22	23	129	63	12	30
disc2	4	219	25	48	800	–	–	28	3
discs	10	–	40	269	–	–	422	59	3
dixchlng	11	12	12	11	31	–	44	26	43
djtl	–	70	29	861	–	–	100	132	1
dnieper	4	35	4	31	13	36	75	25	6
egl	8	8	8	8	9	14	9	10	13
eigencco	20	13	29	14	34	158	17	22	26
eigmaxc	1	9	7	7	18	–	21	14	3
eigminc	1	11	7	8	21	182	11	13	4
engval2	19	21	20	33	34	66	30	28	33
errinros	53	49	53	70	267	–	76	60	90
expfita	13	34	13	31	24	28	54	23	14
expfit	13	9	13	9	18	28	11	13	19
extrosnb	2	2	2	1	2	5	1	1	1

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
fletcher	1	29	1	28	2	–	28	14	19
genhumps	139	257	188	234	77	169	134	146	71
gigomez1	8	19	8	19	9	51	33	16	14
gottfr	6	8	13	9	–	1	35	12	3
gridnetg	5	17	4	11	22	42	21	12	21
gridneth	5	7	5	7	36	114	20	12	12
gridneti	5	18	5	14	47	111	22	16	24
growthls	101	101	106	170	184	258	178	122	212
growth	101	–	106	157	187	258	178	143	206
gulf	26	28	26	44	66	695	63	26	32
hadamals	13	104	13	138	19	266	20	13	38
haifas	12	36	13	10	28	59	27	13	23
hairy	45	51	84	115	35	59	102	64	92
haldmads	11	103	41	259	71	94	45	31	12
hart6	15	10	11	14	16	36	9	23	20
hatflda	10	11	15	11	30	44	47	9	114
hatfldb	9	11	11	11	28	32	28	11	138
hatfldd	31	23	23	27	29	48	66	25	37
hatflde	36	28	26	32	31	63	57	29	31
hatfldf	4	3151	15	1335	–	2	113	13	3
hatfldg	4	20	15	20	–	2	29	16	3
heart6ls	2596	985	–	1544	–	–	–	–	3067
heart6	1	1516	16	75	–	49	–	327	3
heart8ls	248	108	217	189	–	474	238	108	369
heart8	3	37	12	40	–	–	359	39	3
helix	18	13	19	25	28	53	18	13	21
himmelbb	62	15	25	12	8	20	11	16	32
himmelbc	5	8	8	9	–	1	11	8	3
himmelbd	4	7327	4	79	12	–	–	–	3
himmelbe	3	3	2	3	–	1	8	8	2
himmelbf	8	12	8	11	53	47	29	24	15
himmelbg	10	10	10	14	12	17	16	7	13
himmelbh	8	20	8	24	10	10	7	9	10
himmelbk	6	21	6	19	82	109	206	23	11
himmelp1	9	12	9	12	19	20	29	11	21
himmelp2	9	22	9	19	32	151	275	20	18

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
himmelp3	5	10	5	13	8	119	870	17	9
himmelp4	5	10	5	25	8	115	737	17	9
himmelp5	12	25	12	158	44	75	273	90	23
himmelp6	2	11	2	12	2	5	2	45	1
hong	8	16	5	13	4	14	6	20	4
hs001	33	34	36	53	48	9	41	35	36
hs002	9	14	9	17	15	12	7	21	10
hs004	3	6	3	6	4	6	2	8	2
hs005	8	9	11	9	9	13	9	10	14
hs006	3	3	3	7	9	90	63	10	4
hs007	13	65	13	28	30	64	26	12	13
hs008	6	6	6	6	–	1	13	8	3
hs009	5	5	5	6	10	11	22	9	8
hs010	10	26	10	13	20	22	18	15	19
hs011	7	17	6	9	15	46	16	12	12
hs012	9	16	8	9	11	158	26	11	16
hs013	26	45	34	78	17	53	60	–	19
hs014	7	12	6	8	10	9	13	11	3
hs015	3	25	7	21	11	85	47	31	3
hs016	6	12	5	23	5	9	19	18	3
hs017	9	27	8	18	19	11	20	30	6
hs018	8	18	7	27	32	93	117	15	16
hs019	7	18	7	16	9	56	45	18	9
hs020	5	14	5	7	5	8	23	24	3
hs022	6	11	2	7	7	47	10	9	3
hs023	7	15	7	12	7	53	51	18	6
hs024	3	11	3	12	8	8	14	13	3
hs025	31	40	27	44	2	5	1	20	1
hs026	18	25	18	26	27	76	41	14	33
hs027	23	70	8	151	21	136	31	16	43
hs029	9	15	8	9	14	173	18	10	13
hs030	2	26	2	26	5	29	8	9	20
hs031	7	13	6	8	11	28	12	17	20
hs032	2	16	2	20	5	14	7	24	3
hs033	6	14	5	16	9	38	9	11	3

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipop	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
hs034	9	13	8	10	5	9	21	15	16
hs036	3	13	3	13	10	8	7	20	3
hs037	6	13	6	13	10	16	13	11	15
hs038	53	58	54	78	101	88	56	13	94
hs039	19	31	23	14	19	34	22	14	17
hs040	6	6	5	4	9	21	11	8	11
hs041	3	9	2	11	7	5	7	16	14
hs042	7	10	6	7	10	21	13	9	13
hs043	10	15	11	10	11	102	25	12	27
hs045	2	24	2	48	2	5	1	25	1
hs046	19	24	19	20	32	121	28	18	37
hs047	21	22	21	21	28	125	29	24	31
hs049	17	21	17	20	34	61	38	21	21
hs050	9	10	9	10	20	24	11	16	14
hs055	2	8	2	8	2	5	7	11	1
hs056	3	18	19	39	52	52	12	12	27
hs057	5	24	5	28	–	28	2	14	28
hs059	10	91	11	72	22	127	340	27	22
hs060	8	7	7	8	13	120	18	9	23
hs061	1	11	1	10	39	70	19	10	2
hs062	8	10	10	9	16	18	37	13	19
hs063	8	11	1	8	18	119	21	8	19
hs064	11	19	13	18	28	98	38	26	20
hs065	6	15	5	91	11	319	42	19	22
hs066	9	11	14	8	8	8	11	15	15
hs067	13	18	12	12	32	60	287	17	19
hs070	40	18	42	36	34	66	39	23	31
hs071	7	13	6	9	8	55	16	13	14
hs072	14	18	15	17	27	52	65	28	20
hs073	4	12	4	9	11	8	18	21	5
hs074	6	15	6	10	15	27	13	16	15
hs075	5	15	5	10	12	18	110	18	6
hs077	12	12	14	13	16	123	27	12	28
hs078	6	6	5	5	7	57	12	8	13
hs079	6	6	5	5	12	54	14	7	24

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
hs080	9	9	8	7	9	48	13	10	17
hs081	30	12	38	8	11	55	17	16	17
hs083	6	19	5	15	8	11	16	14	6
hs084	11	23	6	12	58	45	47	44	4
hs085	–	–	–	143	–	–	–	63	7
hs086	5	13	5	11	16	13	15	13	19
hs087	7	33	7	18	16	23	32	25	11
hs088	26	44	19	18	59	66	56	27	17
hs089	21	21	31	38	85	197	61	30	19
hs090	73	46	2	28	55	92	58	29	54
hs091	110	122	337	15	73	215	62	28	12
hs092	83	41	2	25	56	110	58	22	103
hs093	3	–	2	10	33	–	–	13	22
hs095	2	27	3	18	1	1	24	17	4
hs096	2	24	3	24	1	1	23	22	4
hs097	7	24	7	24	13	63	19	19	9
hs098	7	43	7	21	13	47	19	93	9
hs099	–	12	9	7	19	60	–	22	29
hs100lnp	13	26	14	21	33	133	32	12	36
hs100mod	13	30	14	27	32	123	137	15	32
hs100	13	37	13	22	23	129	63	12	30
hs101	16	375	34	273	530	–	–	64	49
hs102	17	60	42	36	238	980	–	154	46
hs103	33	76	28	64	177	1418	–	88	38
hs104	19	15	23	11	29	85	–	14	3
hs105	9	21	9	31	89	114	–	17	24
hs106	16	17	17	15	34	–	–	27	33
hs107	7	17	6	12	14	20	26	35	21
hs108	35	29	36	17	152	164	43	20	32
hs109	6	60	7	44	349	–	–	45	13
hs110	5	7	5	7	11	43	5	8	15
hs111lnp	41	25	31	16	64	388	57	17	30
hs111	41	25	31	16	70	388	46	15	29
hs112	13	18	12	18	35	92	47	19	58
hs113	6	16	6	12	28	146	97	17	30
hs114	1	–	1	75	9	–	664	–	4
hs116	12	29	14	26	75	52	–	24	25
hs117	7	25	6	23	20	157	66	19	46

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
hs119	8	17	7	15	22	29	28	29	18
hs99exp	13	117	12	30	42	212	–	256	4
hubfit	2	9	2	9	8	11	8	–	–
humps	114	321	–	249	257	193	–	307	259
hypcir	5	6	8	8	–	1	10	8	3
jensmp	11	11	11	10	36	55	10	14	13
kiwcrese	12	26	11	11	13	30	23	14	17
kowosb	18	15	18	23	33	39	24	11	26
lakes	12	18	63	19	39	–	–	288	1
launch	1	–	1	409	246	–	–	–	3
lewispol	1	–	1	–	6	–	–	–	1
loadbal	8	17	8	18	58	130	62	23	26
loghair	90	2071	–	–	249	402	–	64	150
logros	50	95	50	358	109	147	66	398	84
lootsma	6	14	5	16	9	38	9	12	3
lsnnodoc	7	16	7	15	8	7	11	21	3
madsen	14	26	25	25	14	28	26	26	28
makela1	14	20	15	19	8	27	19	15	20
makela2	5	11	5	8	16	21	40	12	7
makela3	22	35	25	17	288	96	125	18	15
maratos	11	42	10	5	9	20	9	7	11
matrix2	23	45	12	21	14	65	13	26	161
maxlika	9	21	9	31	89	114	–	17	24
mconcon	3	10	5	10	14	–	676	–	40
mdhole	3	70	56	106	70	116	68	98	6
methanb8	47	8	47	9	306	175	221	155	28
methanl8	105	66	96	82	499	669	640	80	84
mexhat	10	5	10	5	37	18	4	7	8
meyer3	–	–	280	492	–	775	559	522	716
mifflin1	11	24	23	7	10	15	18	9	8
mifflin2	11	30	10	16	14	53	50	13	13
minmaxbd	44	88	9	78	115	160	592	31	38
minmaxrb	3	11	3	11	4	34	81	14	5
minsurf	14	13	10	21	24	196	15	13	15
mistake	22	30	18	15	19	159	30	15	36
mwright	10	10	12	11	12	37	19	11	19
nonmsqrt	–	–	716	–	–	1490	167	–	–
nuffield	5	7	5	7	9	15	15	10	17
odfits	5	11	7	11	17	28	49	15	16
optcntrl	2	192	4	134	5	15	352	58	6

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
optmass	9	26	18	23	2	331	–	15	27
optprloc	16	23	6	19	12	685	438	23	19
orthregb	2	3	2	3	9	262	64	8	2
orthrege	7995	41	180	166	31	857	795	482	13038
osbornea	4110	50	–	152	120	127	57	–	52
osborneb	20	21	20	25	82	127	45	19	42
palmer1a	42	59	51	71	205	–	102	113	200
palmer1b	21	21	21	26	87	–	55	65	69
palmer1e	105	49	74	153	186	149	353	187	100
palmer1	33	1129	33	1844	30	39	28	41	21
palmer2a	68	189	68	261	115	197	211	169	83
palmer2b	16	21	16	34	61	–	77	52	49
palmer2e	99	50	86	52	191	345	133	171	111
palmer2	32	3257	33	63	44	–	34	24	57
palmer3a	78	759	82	200	136	–	201	172	172
palmer3b	21	18	21	15	54	–	36	37	49
palmer3e	449	112	117	116	293	426	–	234	118
palmer3	11	348	12	537	13	11	58	32	23
palmer4a	52	174	52	133	109	–	93	110	42
palmer4b	21	19	21	31	52	–	64	35	48
palmer4e	25	39	24	38	123	210	123	87	65
palmer4	12	363	12	1092	14	11	142	35	32
palmer5a	7809	10195	–	–	–	321078	–	–	–
palmer5b	837	97	855	171	–	3728	961	–	–
palmer5e	3	5877	3	–	–	25501	8	–	–
palmer6a	132	378	137	275	202	270	277	–	–
palmer6e	22	39	38	59	198	259	47	–	–
palmer7a	–	5811	–	–	–	–	–	–	–
palmer7e	1526	9348	–	–	–	924	39	–	–
palmer8a	49	105	50	102	127	103	61	–	–
palmer8e	29	31	29	31	92	121	86	–	–
pentagon	15	19	12	19	15	22	48	38	23
pfit1ls	365	395	566	682	480	717	453	337	1064
pfit1	365	395	566	682	480	717	453	337	1064
pfit2ls	178	168	210	172	175	237	217	115	2586
pfit2	178	168	210	172	175	237	217	115	2586
pfit3ls	166	162	139	347	289	474	272	148	3428
pfit3	166	162	139	347	289	474	272	148	3428
pfit4ls	84	286	126	532	470	754	410	286	4925
pfit4	84	286	126	532	470	754	410	286	4925

Table 5 continued

Problem	Uno presets		State-of-the-art solvers						
	filtersqp	ipopt	filterSQP	IPOPT	SNOPT	MINOS	LANCELOT	LOQO	CONOPT
polak1	9	24451	8	7	15	30	37	14	24
polak2	45	119	10	15	106	138	321	24	2
polak3	21	–	24	–	183	–	234	24	2
polak4	5	175	5	7	6	22	16	11	37
polak5	81	55	45	33	43	16	7	69	17
polak6	27	–	29	660	62	108	644	27	40
powellbs	9	221	16	12	–	1	–	17	3
powellsq	–	160	4	109	78	–	23	–	2
prodp10	10	20	9	16	60	42	35	24	34
prodp11	8	22	7	17	59	56	27	21	5
pspdoc	9	11	7	15	15	25	10	12	12
recipe	3	3	2	3	–	1	12	10	2
rk23	7	14	9	12	18	28	54	12	6
robot	1	19	45	10	18	377	33	18	64
rosenbr	29	29	29	45	45	9	36	26	29
rosenmmx	31	71	37	22	41	66	226	17	75
s365mod	19	38	86	43	31	539	–	28	57
sineali	10	5792	–	–	–	7333	–	16	24
sineval	57	66	62	110	94	123	75	57	77
sisser	19	21	19	21	15	17	38	17	19
snake	3	10	3	14	2	9	–	–	4
spanhyd	6	–	11	26	13	–	26	–	24
spiral	121	5743	152	64	130	148	96	135	642
ssnlbeam	6	28	5	22	36	124	39	60	44
stancmin	2	11	2	11	5	5	9	20	4
swopf	6	24	6	17	160	100	290	21	22
synthes1	6	12	5	10	10	21	13	17	13
try-b	9	11	8	20	10	10	13	16	3
twobars	9	13	8	10	11	31	13	10	14
vander4	1	–	1	248	86	–	36	–	3
watson	22	14	21	14	172	117	44	18	18
weeds	38	26	39	32	51	–	3	29	44
womflet	17	110	9	12	14	40	97	12	25
yfit	47	60	48	191	95	130	103	45	74
yfitu	47	45	48	69	95	130	103	43	74
zecevic3	10	23	9	22	11	38	19	12	19
zecevic4	7	13	6	10	8	22	12	15	16
zigzag	12	28	11	23	23	218	43	29	74
zy2	6	14	5	10	9	43	9	14	4

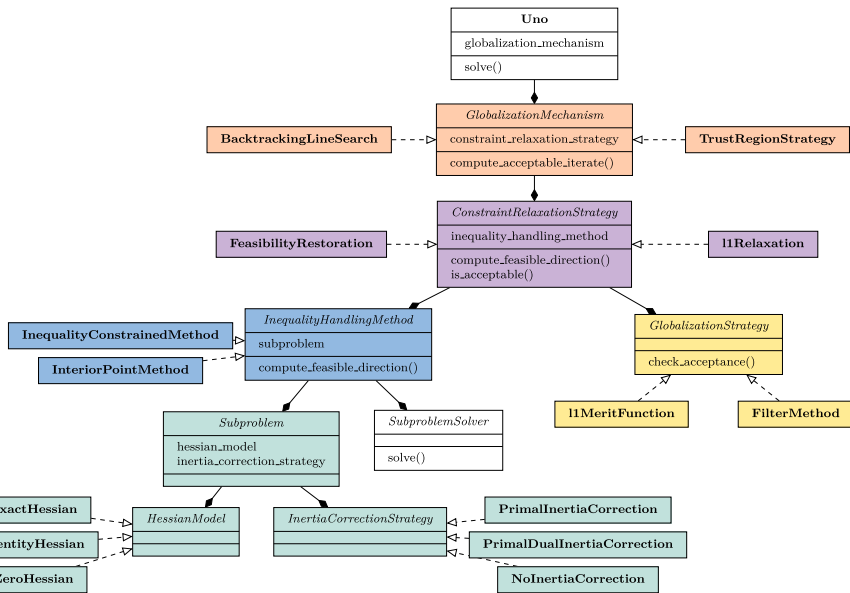


Fig. 5 Uno's UML diagram

8 Conclusion and future developments

We have introduced an abstract framework for unifying nonlinearly constrained optimization with eight key ingredients common to most methods. We have shown that our abstract framework provides common notation and abstractions for the unified description of well-known optimization strategies. We then presented Uno, a C++ implementation of the unifying framework. A modular solver, Uno provides efficient implementations of off-the-shelf strategies and facilitates the development of new algorithmic ideas by making building blocks and abstractions readily available. In particular, newly developed strategies (e.g., a new globalization strategy) can be immediately deployed and tested within a wide range of strategy combinations. Strategy combinations that do not exist as software implementations can be generated on the fly with no programming effort and tested on a given instance. We believe that Uno has the potential to serve as an experimentation laboratory for the optimization community and accelerate research in nonlinearly constrained optimization.

Future releases of Uno will include the following features: quasi-Newton methods (L-BFGS and SR1), iterative linear solvers (e.g., MINRES [41]), equality-constrained methods (SLP-EQP, SQUID's Sl_1 QP method [6], parallel line search [44], and additional interfaces to subproblem solvers.

Appendices

Table 6 Description of state-of-the-art solvers within the proposed unifying framework.

Solver	Constraint relaxation strategy	Inequality handling method	Globalization strategy	Globalization mechanism	Hessian model	Inertia correction strategy	Method
ALGLIB MinNLC	ℓ_1 relaxation	IC	filter method	trust region	QN	none	SQP
CONOPT	feasible step method	IC	objective merit	line search	QN	none	GRG
FICO XSLP	ℓ_1 relaxation	IC	–	ℓ_∞ trust region	none	none	SLP
filterSQP	feasibility restoration	IC	filter method	ℓ_∞ trust region	exact	none	SQP
IPOPT	feasibility restoration	primal-dual IPM	filter method	line search	exact / QN	primal-dual	IPM
KNITRO-ASM	ℓ_1 relaxation	EC	ℓ_1 merit	trust region	exact	?	SLQP
KNITRO-IPM	step decomposition	primal-dual IPM	ℓ_2 merit	line search + trust region	exact	none	IPM
LANCELOT	bound-constrained	AL EC	AL merit	ℓ_2 / ℓ_∞ trust region	exact	none	AL
LOQO	ℓ_2^2 relaxation	primal-dual IPM	ℓ_2^2 merit	line search	exact	primal	IPM
MINOS	linearly constrained	IC	forcing sequences	line search	QN	none	GRG
NAG e04uc/e04wdc	AL + ℓ_1 relaxation						
NLPQL	ℓ_1 relaxation	IC	AL merit	line search	QN	none	SQP
	right-hand-side relaxation	IC	AL merit	line search	QN	none	SQP
SLSQP	right-hand-side relaxation	IC	ℓ_1 merit	line search	QN	none	SQP
SNOPT	ℓ_1 relaxation	IC	AL merit	line search	QN	none	SQP
SQUID	ℓ_1 relaxation	IC	ℓ_1 merit	line search	exact	primal	SL ₁ QP
WORHP	right-hand-side relaxation	IC	ℓ_1 merit / filter method	line search	QN	none	SQP

Algorithm 2: Uno: trust-region filter restoration SQP.

Input: initial primal-dual iterate $(x^{(0)}, y^{(0)}, z^{(0)})$, initial trust-region radius Δ
 $k \leftarrow 0$

$(\eta(x), \omega_x(x), \xi(x)) \stackrel{\text{def}}{=} (\|c(x)\|_1, f(x), 0)$ *constraint relaxation*

$(\Delta\eta^{(k)}(d_x), \Delta\omega_x^{(k)}(d_x), \Delta\xi^{(k)}(d_x)) \stackrel{\text{def}}{=} (\|c^{(k)}\|_1 - \|\nabla c^{(k)}\|^T d_x, -(\nabla f^{(k)})^T d_x, 0)$

$\text{phase} \leftarrow \text{Optimality}$

$\phi \stackrel{\text{def}}{=} \omega_1 + \xi$ *globalization strategy*

$\Delta\phi^{(k)} \stackrel{\text{def}}{=} \Delta\omega_1^{(k)} + \Delta\xi^{(k)}$

Initialize $\mathcal{F}$

repeat

Set inner iteration counter $l \leftarrow 0$ *globalization mechanism*

Reset trust-region radius $\Delta^{(l)} \in [\underline{\Delta}, \bar{\Delta}]$

repeat

if $\text{phase} = \text{Optimality}$ then *constraint relaxation*

$(d_x^{(k,l)}, d_y^{(k,l)}, d_z^{(k,l)}) \leftarrow \text{solve } QP^{(k)}(\Delta^{(l)})$

if $QP^{(k)}(\Delta^{(l)})$ infeasible then

$\text{phase} \leftarrow \text{Restoration}$

$y^{(k)} \leftarrow 0$

$\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$ *globalization strategy*

if $\text{phase} = \text{Restoration}$ then

$(d_x^{(k,l)}, d_y^{(k,l)}, d_z^{(k,l)}) \leftarrow \text{solve } FQP^{(k)}(\Delta^{(l)})$ starting from $d_z^{(k,l)}$

Assemble trial iterate $(\tilde{x}^{(k+1,l)}, \tilde{y}^{(k+1,l)}, \tilde{z}^{(k+1,l)}) \stackrel{\text{def}}{=} (x^{(k)}, y^{(k)}, z^{(k)}) + (d_x^{(k,l)}, d_y^{(k,l)}, d_z^{(k,l)})$

Reset the bound multipliers corresponding to the active trust region

$\text{acceptable} \leftarrow \text{false}$

if $\|d_x^{(k,l)}\| = 0$ then *constraint relaxation*

$\text{acceptable} \leftarrow \text{true}$

else

if $\text{phase} = \text{Restoration}$ and $QP^{(k)}(\Delta^{(l)})$ feasible and $\eta(\tilde{x}^{(k+1,l)}) < \eta_{\min}(\mathcal{F})$ then

$\text{phase} \leftarrow \text{Optimality}$

$\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$

if $\text{phase} = \text{Restoration}$ then *globalization strategy*

if $\Delta\eta(x^{(k)}) - \eta(\tilde{x}^{(k+1,l)}) \geq \sigma \Delta\eta^{(k)}(d_x^{(k,l)})$ then

$\text{acceptable} \leftarrow \text{true}$

else if $\tilde{z}^{(k+1,l)}$ acceptable to $\mathcal{F}$ and improves upon $x^{(k)}$ then

if $\Delta\phi^{(k)}(d_x^{(k,l)}) \geq \delta\eta(x^{(k)})^2$ then

if $\phi(x^{(k)}) - \phi(\tilde{x}^{(k+1,l)}) \geq \sigma \Delta\phi^{(k)}(d_x^{(k,l)})$ then

$\text{acceptable} \leftarrow \text{true}$ (f-type)

else

$\text{acceptable} \leftarrow \text{true}$ (h-type)

$\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$

if acceptable then

if trust region is active at $d_x^{(k,l)}$ then

Increase radius $\Delta^{(l)}$

else

Decrease radius $\Delta^{(l)}$

$l \leftarrow l + 1$

until $(\tilde{x}^{(k+1,l)}, \tilde{y}^{(k+1,l)}, \tilde{z}^{(k+1,l)})$ is acceptable

Update $(x^{(k+1)}, y^{(k+1)}, z^{(k+1)}) \leftarrow (\tilde{x}^{(k+1,l)}, \tilde{y}^{(k+1,l)}, \tilde{z}^{(k+1,l)})$

$k \leftarrow k + 1$

until termination criteria are satisfied

return $(x^{(k)}, y^{(k)}, z^{(k)})$

Algorithm 3: Uno: line-search filter restoration interior-point method.

Input: initial primal-dual iterate $(x^{(0)}, y^{(0)}, z^{(0)})$, initial barrier parameter $\mu > 0$
 $k \leftarrow 0$

$(\eta(x), \omega_\pi(x), \xi(x)) \stackrel{\text{def}}{=} (\|c(x)\|_1, f(x), -\mu \log(X)e)$ *constraint relaxation*

$(\Delta \eta^{(k)}(d_x), \Delta \omega_\pi^{(k)}(d_x), \Delta \xi^{(k)}(d_x)) \stackrel{\text{def}}{=} \left(\|c^{(k)}\|_1 - \|c^{(k)} + (\nabla c^{(k)})^T d_x\|_1, -(\nabla f^{(k)})^T d_x, \mu(X^{(k)})^{-1} e^T d_x \right)$

$\text{phase} \leftarrow \text{Optimality}$

$\phi \stackrel{\text{def}}{=} \omega_1 + \xi$ *globalization strategy*

$\Delta \phi^{(k)} \stackrel{\text{def}}{=} \Delta \omega_1^{(k)} + \Delta \xi^{(k)}$

Initialize $\mathcal{F}$

repeat

Possibly update the barrier parameter μ *subproblem*

$(d_x^{(k)}, d_y^{(k)}, d_z^{(k)}) \leftarrow \text{solve } \begin{cases} I P S P_\mu^{(k)} & \text{if phase = Optimality} \\ F I P S P_\mu^{(k)} & \text{if phase = Restoration} \end{cases}$

Scale primal-dual direction according to (Eq. 10) *subproblem*

$\alpha^{(0)} \leftarrow 1$
 Set inner iteration counter $l \leftarrow 0$

repeat

Assemble trial iterate $(\hat{x}^{(k+1,l)}, \hat{y}^{(k+1,l)}, \hat{z}^{(k+1,l)}) \stackrel{\text{def}}{=} (x^{(k)}, y^{(k)}, z^{(k)}) + (\alpha^{(l)} d_x^{(k)}, \alpha^{(l)} d_y^{(k)}, \alpha^{(l)} d_z^{(k)})$

$\text{acceptable} \leftarrow \text{false}$

if $\|d^{(k)}\| = 0$ **then** *constraint relaxation*
 $\text{acceptable} \leftarrow \text{true}$

else *globalization strategy*

if phase = Restoration **then**
 if $\eta(x^{(k)}) - \eta(\hat{x}^{(k+1,l)}) \geq \sigma \Delta \eta^{(k)}(d_x^{(k)})$ **then**
 $\text{acceptable} \leftarrow \text{true}$

else if $\hat{x}^{(k+1,l)}$ acceptable to $\mathcal{F}$ **then**
 if $\eta(x^{(k)}) \leq \theta_{\min}$ and $0 < \Delta \phi^{(k)}(\alpha^{(l)} d_x^{(k)})$ and $\Delta \phi^{(k)}(\alpha^{(l)} d_x^{(k)}) \geq \delta \eta(x^{(k)})^2$ **then**
 if $\phi(x^{(k)}) - \phi(\hat{x}^{(k+1,l)}) \geq \sigma \Delta \phi^{(k)}(\alpha^{(l)} d_x^{(k)})$ **then**
 $\text{acceptable} \leftarrow \text{true}$
 $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$

else if $\hat{x}^{(k+1,l)}$ improves upon $x^{(k)}$ **then**
 $\text{acceptable} \leftarrow \text{true}$
 if $\neg (0 < \Delta \phi^{(k)}(\alpha^{(l)} d_x^{(k)}) \text{ and } \Delta \phi^{(k)}(\alpha^{(l)} d_x^{(k)}) \geq \delta \eta(x^{(k)})^2)$ **then**
 $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$

if acceptable and phase = Restoration and $\hat{x}^{(k+1,l)}$ acceptable to $\mathcal{F}$ and $\eta(\hat{x}^{(k+1,l)}) \leq \kappa \eta(x^{(k)})$ **then**
 phase $\leftarrow$ Optimality
 $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$

if not acceptable then
 Decrease step length $\alpha^{(l)}$

$l \leftarrow l + 1$

if $\alpha^{(l)}$ too small **then**

 phase $\leftarrow$ Restoration *constraint relaxation*
 $\mathcal{F} \leftarrow \mathcal{F} \cup \{(\eta(x^{(k)}), \phi(x^{(k)}))\}$ *globalization strategy*

$(d_x^{(k)}, d_y^{(k)}, d_z^{(k)}) \leftarrow \text{solve } F I P S P_\mu^{(k)}$

Scale primal-dual direction according to (Eq. 10) *subproblem*

until $(\hat{x}^{(k+1,l)}, \hat{y}^{(k+1,l)}, \hat{z}^{(k+1,l)})$ is acceptable

Update $(x^{(k+1)}, y^{(k+1)}, z^{(k+1)}) \leftarrow (\hat{x}^{(k+1,l)}, \hat{y}^{(k+1,l)}, \hat{z}^{(k+1,l)})$

$k \leftarrow k + 1$

until termination criteria are satisfied

return $(x^{(k)}, y^{(k)}, z^{(k)})$

Acknowledgements This work was supported by the Applied Mathematics activity within U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research, under Contract DE-AC02-06CH11357. We would like to thank David Kiessling (KU Leuven) for fruitful discussions about constrained optimization and his extensive testing of Uno, Nils-Christian Kempke (Cardinal Operations) for his valuable help with technical questions in C++, and Gail Pieper (Argonne National Laboratory) for proofreading our manuscript. We are highly grateful to the following individuals who contributed their expertise and time to the open-source implementation of Uno: Oscar Dowson, Marcel Jacobse, David Kiessling, Rujia Liu, Stefano Lovato, Alexis Montoison, Manuel Schaich, and Silvio Traversaro. We have greatly benefited from their careful contributions and discussions.

Funding This work was supported by the Applied Mathematics activity within U.S. Department of Energy, Office of Science, Advanced Scientific Computing Research, under Contract DE-AC02-06CH11357.

Data Availability The benchmark data used for the solver comparison is available here: https://github.com/cvanaret/nonlinear_optimization_solver_benchmark.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. NAG library manual, mark 26. Tech. rep., The Numerical Algorithms Group, https://support.nag.com/numeric/cl/nagdoc_cl26/pdf/frontmatter/manconts.pdf (2017)
2. FICO Xpress nonlinear manual. Tech. rep., FICO, <https://www.fico.com/fico-xpress-optimization/docs/latest/solver/nonlinear/HTML> (2023)
3. Amestoy, P.R., Duff, I.S., L'Excellent, J.Y., Koster, J.: MUMPS: a general purpose distributed memory sparse solver. In: International Workshop on Applied Parallel Computing, pp. 121–130. Springer (2000)
4. Bochkkanov, S., Bystritsky, V.: ALGLIB - a Cross-Platform Numerical Analysis and Data Processing Library. ALGLIB Project. Novgorod, Russia (2011)
5. Bongartz, I., Conn, A.R., Gould, N., Toint, P.L.: CUTE: Constrained and unconstrained testing environment. ACM Trans. Math. Softw. (TOMS) **21**(1), 123–160 (1995)
6. Burke, J.V., Curtis, F.E., Wang, H.: A sequential quadratic optimization algorithm with rapid infeasibility detection. SIAM J. Optim. **24**(2), 839–872 (2014)
7. Büskens, C., Wassel, D.: The ESA NLP solver WORHP. Modeling and optimization in space engineering pp. 85–110 (2013)
8. Byrd, R., Nocedal, J., Waltz, R.: KNITRO: An integrated package for nonlinear optimization. In: di Pillo, G., Roma, M. (eds.) Large-Scale Nonlinear Optimization, pp. 35–59. Springer-Verlag, New York (2006)
9. Byrd, R.H., Curtis, F.E., Nocedal, J.: Infeasibility detection and SQP methods for nonlinear optimization. SIAM J. Optim. **20**(5), 2281–2299 (2010)
10. Byrd, R.H., Nocedal, J., Waltz, R.A.: Steering exact penalty methods for nonlinear programming. Optim. Methods Softw. **23**(2), 197–213 (2008)
11. Chin, C., Fletcher, R.: On the global convergence of an SLP-filter algorithm that takes EQP steps. Math. Program. **96**(1), 161–177 (2003)

12. Conn, A., Gould, N., Toint, P.: LANCELOT: A Fortran Package for Large-Scale Nonlinear Optimization (Release A). Springer Verlag, Heidelberg, New York (1992)
13. Dolan, E.D., Moré, J.: Benchmarking optimization software with performance profiles. *Math. Program.* **91**(2), 201–213 (2002)
14. Drud, A.S.: CONOPT - a large-scale GRG code. *ORSA J. Comput.* **6**(2), 207–216 (1994)
15. Duff, I.S.: MA57-a code for the solution of sparse symmetric definite and indefinite systems. *ACM Trans. Math. Softw. (TOMS)* **30**(2), 118–144 (2004)
16. Duff, I.S., Gould, N.I., Reid, J.K., Scott, J.A., Turner, K.: The factorization of sparse symmetric indefinite matrices. *IMA J. Numer. Anal.* **11**(2), 181–204 (1991)
17. Fletcher, R.: Stable reduced Hessian updates for indefinite quadratic programming. *Math. Program.* **87**(2), 251–264 (2000)
18. Fletcher, R.: *Practical Methods of Optimization*. John Wiley & Sons, USA (2013)
19. Fletcher, R., Gould, N.I., Leyffer, S., Toint, P.L., Wächter, A.: Global convergence of a trust-region SQP-filter algorithm for general nonlinear programming. *SIAM J. Optim.* **13**(3), 635–659 (2002)
20. Fletcher, R., Leyffer, S.: User manual for filterSQP. Numerical Analysis Report NA/181, University of Dundee (1998)
21. Fletcher, R., Leyffer, S.: Solving mathematical program with complementarity constraints as nonlinear programs. *Optim. Methods Softw.* **19**(1), 15–40 (2004)
22. Fletcher, R., Leyffer, S., Toint, P.: On the global convergence of a filter-SQP algorithm. *SIAM J. Optimization* **13**(1), 44–59 (2002)
23. Fletcher, R.: An optimal positive definite update for sparse Hessian matrices. *SIAM J. Optim.* **5**(1), 192–218 (1995)
24. Fourer, R., Gay, D.M., Kernighan, B.W.: *AMPL: A Modelling Language for Mathematical Programming*, 2nd edn. Books/Cole—Thomson Learning (2003)
25. Fowkes, J.M., et al.: Galahad 4.0: an open source library of fortran packages with c and matlab interfaces for continuous optimization. *J. Open Source Softw.* **8**(87), 4882 (2023)
26. Gay, D.M.: Hooking your solver to AMPL. Tech. rep., Technical Report 93-10, AT&T Bell Laboratories, Murray Hill, NJ, 1993, revised (1997)
27. Gill, P., Murray, W., Saunders, M.: SNOPT: An SQP algorithm for large-scale constrained optimization. *SIAM J. Optim.* **12**(4), 979–1006 (2002)
28. Gould, N.M., Leyffer, S.: An introduction to algorithms for nonlinear optimization. In: Blowey, J., Craig, A., Shardlow, T. (eds.) *Frontiers in Numerical Analysis*, pp. 109–197. Springer Verlag, Berlin (2003)
29. Han, S.: A globally convergent method for nonlinear programming. *J. Optim. Theory Appl.* **22**(3), 297–309 (1977)
30. Huangfu, Q., Hall, J.J.: Parallelizing the dual revised simplex method. *Math. Program. Comput.* **10**(1), 119–142 (2018)
31. Kraft, D.: A software package for sequential quadratic programming. *Forschungsbericht- Deutsche Forschungs- und Versuchsanstalt für Luft- und Raumfahrt* (1988)
32. Leyffer, S.: Complementarity constraints as nonlinear equations: Theory and numerical experience. In: S. Dempe, V. Kalashnikov (eds.) *Optimization and Multivalued Mappings*, pp. 169–208. Springer (2006)
33. Leyffer, S., Lopez-Calva, G., Nocedal, J.: Interior methods for mathematical programs with complementarity constraints. *SIAM J. Optim.* **17**(1), 52–77 (2006)
34. Leyffer, S., Mahajan, A.: Software for nonlinearly constrained optimization. *Wiley Encyclopedia of Operations Research and Management Science* (2010)
35. Leyffer, S., Menickelly, M., Munson, T., Vanaret, C., Wild, S.M.: A survey of nonlinear robust optimization. *INFOR: Information Systems and Operational Research* (2020). <https://doi.org/10.1080/03155986.2020.1730676>
36. Luo, Z.Q., Pang, J.S., Ralph, D.: *Mathematical Programs with Equilibrium Constraints*. Cambridge University Press, Cambridge, UK (1996)
37. Mangasarian, O.L., Fromovitz, S.: The Fritz John necessary optimality conditions in the presence of equality and inequality constraints. *J. Math. Anal. Appl.* **17**(1), 37–47 (1967)
38. Murtagh, B., Saunders, M.: MINOS 5.4 user's guide. Report SOL 83-20R, Department of Operations Research, Stanford University (1993)
39. Nocedal, J., Wright, S.J.: *Numerical Optimization*. Springer (2006)

40. Outrata, J., Kocvara, M., Zowe, J.: *Nonsmooth Approach to Optimization Problems with Equilibrium Constraints*. Kluwer Academic Publishers, Dordrecht (1998)
41. Paige, C.C., Saunders, M.A.: Solution of sparse indefinite systems of linear equations. *SIAM J. Numer. Anal.* **12**(4), 617–629 (1975)
42. Powell, M.J.D.: A fast algorithm for nonlinearly constrained optimization calculations. In: Watson, G. (ed.) *Numerical Analysis*, 1977, pp. 144–157. Springer-Verlag, Berlin (1978)
43. Powell, M.J.D., Yuan, Y.: A recursive quadratic programming algorithm that uses differentiable exact penalty functions. *Math. Program.* **35**, 165–278 (1986)
44. Schittkowski, K.: *NLPQLP: A new Fortran implementation of a sequential quadratic programming algorithm for parallel computing*. Report, Department of Mathematics, University of Bayreuth (2001)
45. Vanaret, C., Leyffer, S.: *Argonot: An open-source software framework for nonlinear optimization* (2018). 23rd International Symposium on Mathematical Programming (ISMP 2018)
46. Vanderbei, R.J.: LOQO: An interior point code for quadratic programming. *Optim. Methods Softw.* **11**(1–4), 451–484 (1999)
47. Wächter, A., Biegler, L.: Line search filter methods for nonlinear programming: Local convergence. *SIAM J. Optim.* **16**(1), 32–48 (2005a)
48. Wächter, A., Biegler, L.: Line search filter methods for nonlinear programming: Motivation and global convergence. *SIAM J. Optim.* **16**(1), 1–31 (2005b)
49. Wächter, A., Biegler, L.T.: On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Math. Program.* **106**(1), 25–57 (2006)
50. Wilson, R.: *A simplicial algorithm for concave programming*. Ph.D. thesis, Harvard University Graduate School of Business Administration (1963)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.